🐍 Python Programming Course

📘 Day 8: Tuples in Python

Learn Python from Beginner to Advanced

📚 Topics Covered

- What are Tuples?
- Why Do We Need Tuples?
- Characteristics of Tuples
- Advantages of Tuples
- Tuple vs List
- When Should You Use a Tuple?
- Creating Tuples
- Accessing Tuple Elements
- Tuple Slicing
- Tuple Immutability
- Updating Tuples (Indirect Method)
- Tuple Packing & Unpacking
- Extended Unpacking (`*`)
- Looping Through Tuples
- Tuple Operations
- Built-in Functions
- Tuple Methods
- Copying Tuples
- Joining Tuples
- Nested Tuples
- Tuple Memory Efficiency
- Tuple Performance
- Tuples as Dictionary Keys
- Returning Multiple Values Using Tuples
- Real-World Applications
- Common Errors
- Best Practices
- Coding Questions
- Practice Programs
- Cheat Sheet
- Summary

🎯 Learning Objectives

By the end of this lesson, you will be able to:

- ✅ Understand what tuples are and when to use them.

- ✅ Explain the characteristics and advantages of tuples.
- ✅ Differentiate between tuples and lists.
- ✅ Create tuples using different techniques.
- ✅ Access tuple elements using indexing and slicing.
- ✅ Understand tuple immutability and modify tuples using indirect methods.
- ✅ Pack and unpack tuples, including extended unpacking (`*`).
- ✅ Iterate through tuples using different looping techniques.
- ✅ Perform tuple operations such as concatenation, repetition, and membership testing.
- ✅ Use built-in functions like `len()` , `min()` , `max()` , `sum()` , `sorted()` , `any()` , and `all()` with tuples.
- ✅ Use tuple methods such as `count()` and `index()` .
- ✅ Copy and join tuples correctly.
- ✅ Work with nested tuples.
- ✅ Understand tuple memory efficiency and performance benefits.
- ✅ Use tuples as dictionary keys and for returning multiple values from functions.
- ✅ Identify common mistakes when working with tuples.
- ✅ Follow best practices for writing clean and efficient tuple-based programs.
- ✅ Solve real-world programming problems using tuples.

What is a Tuple?

A **tuple** is an **ordered**, **immutable** collection of elements in Python.

It is used to store multiple values in a single variable. Unlike lists, the elements of a tuple **cannot be modified** after the tuple is created.

Tuples can store elements of the **same data type** or **different data types**, and duplicate values are allowed.

> **Note:** Tuples are created using **parentheses** `()` . However, Python also recognizes a tuple when values are separated by commas.

Syntax

```
tuple_name = (item1, item2, item3)
```

Example 1: Creating a Tuple

In [ ]:
```
fruits = ("Apple", "Banana", "Mango")

print(fruits)
print(type(fruits))
```

Output

```text
('Apple', 'Banana', 'Mango')
```

Explanation

The tuple `fruits` contains three string elements.

Example 2: Tuple with Different Data Types

In [ ]:
```
student = ("Alice", 20, 89.5, True)

print(student)
```

Output

```text
('Alice', 20, 89.5, True)
```

Explanation

A tuple can store different data types such as:

- String
- Integer
- Float
- Boolean

Example 3: Tuple with Duplicate Values

In [ ]:
```
numbers = (10, 20, 10, 30, 20)

print(numbers)
```

Output

```text
(10, 20, 10, 30, 20)
```

Explanation

Tuples allow duplicate values.

Example 4: Tuple Without Parentheses

In [ ]:
```
colors = "Red", "Green", "Blue"
```

```
print(colors)
print(type(colors))
```

Output

```text
('Red', 'Green', 'Blue')
```

Explanation

Python automatically creates a tuple when values are separated by commas, even without parentheses.

Real-Life Example

A person's **date of birth** usually never changes.

In [ ]:
```
date_of_birth = (15, "August", 2005)

print(date_of_birth)
```

Output

```text
(15, 'August', 2005)
```

Since a date of birth is fixed, a tuple is a suitable choice.

Summary Table

Feature	Tuple
Ordered	✅ Yes
Mutable	❌ No (Immutable)
Allows Duplicates	✅ Yes
Indexed	✅ Yes
Stores Mixed Data Types	✅ Yes
Created Using	`()`

Key Points

- A tuple is an ordered collection of elements.
- Tuples are **immutable**, meaning their elements cannot be changed after creation.
- Tuples allow duplicate values.
- Tuples support indexing and slicing.
- Tuples can store elements of different data types.
- Tuples are usually created using parentheses `()`.

Why Do We Need Tuples?

Tuples are used to store a collection of values that **should not change** during the execution of a program.

Unlike lists, tuples are **immutable**, which helps protect data from accidental modification.

Tuples are also more memory-efficient and slightly faster than lists, making them a good choice for storing fixed data.

> **Note:** Use a tuple when the data is fixed and should remain unchanged.

Why Are Tuples Important?

1. Protect Data

Tuples prevent accidental changes to data because they are immutable.

Example

```
days = ("Monday", "Tuesday", "Wednesday")

print(days)
```

Once created, the tuple cannot be modified.

---

2. Store Fixed Information

Tuples are ideal for storing information that rarely or never changes.

Examples include:

- Days of the week
- Months of the year
- Coordinates `(x, y)`
- RGB color values
- Date of birth

Example

```
months = (
    "January", "February", "March",
    "April", "May", "June",
    "July", "August", "September",
    "October", "November", "December"
)

print(months)
```

---

3. Improve Program Performance

Since tuples are immutable, Python can optimize them internally.

As a result, tuples are generally:

- Faster to access
- More memory-efficient than lists

This makes them suitable for large collections of constant data.

---

4. Return Multiple Values from Functions

Functions can return multiple values using tuples.

Example

```
def student():
    return "Alice", 20

name, age = student()

print(name)
print(age)
```

Output

```
Alice
20
```

---

5. Use as Dictionary Keys

Tuples can be used as dictionary keys because they are immutable.

Example

```
locations = {
    (28.6139, 77.2090): "New Delhi",
    (19.0760, 72.8777): "Mumbai"
}

print(locations[(28.6139, 77.2090)])
```

Output

```
New Delhi
```

Real-Life Examples

Student Information

```
student = ("Shivam", 101, "CSE")
```

Student ID and department usually do not change frequently.

GPS Coordinates

```
location = (28.6139, 77.2090)
```

Coordinates represent a fixed location.

RGB Color

```
red = (255, 0, 0)
```

The RGB values for a color remain constant.

When Should You Use a Tuple?

Use a tuple when:

- The data should not be modified.
- The order of elements is important.
- You need better memory efficiency.
- You want slightly faster performance than a list.
- You want to use the collection as a dictionary key.

Key Points

- Tuples store fixed collections of data.
- Tuples are immutable, making them safer for constant values.
- Tuples are more memory-efficient than lists.
- Tuples are slightly faster than lists.
- Tuples are commonly used for coordinates, dates, colors, configuration values, and returning multiple values from functions.

Characteristics of Tuples

A tuple has several important characteristics that make it different from other Python data structures such as lists and sets.

Understanding these characteristics will help you decide when to use a tuple in your programs.

---

1. Ordered Collection

Tuples maintain the order of elements.

This means the elements are stored in the same order in which they are inserted.

Example

```
fruits = ("Apple", "Banana", "Mango")

print(fruits)
```

Output

```
('Apple', 'Banana', 'Mango')
```

Explanation

The elements are displayed in the same order they were added.

---

2. Immutable

Tuples are **immutable**, which means their elements cannot be changed, added, or removed after the tuple is created.

Example

```
numbers = (10, 20, 30)

numbers[0] = 100
```

Result

```
TypeError: 'tuple' object does not support item assignment
```

Explanation

Python raises a `TypeError` because tuple elements cannot be modified.

---

3. Allows Duplicate Values

Tuples can contain duplicate elements.

Example

```
numbers = (10, 20, 10, 30, 20)

print(numbers)
```

Output

```
(10, 20, 10, 30, 20)
```

Explanation

The values `10` and `20` appear more than once.

---

4. Indexed

Each element in a tuple has an index.

Indexing starts from **0**, just like lists.

Example

```
fruits = ("Apple", "Banana", "Mango")

print(fruits[1])
```

Output

```
Banana
```

---

5. Supports Negative Indexing

Tuples also support negative indexes.

Example

```
fruits = ("Apple", "Banana", "Mango")

print(fruits[-1])
```

Output

```
Mango
```

Explanation

`-1` refers to the last element of the tuple.

---

6. Supports Slicing

You can retrieve a portion of a tuple using slicing.

Example

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
```

Output

```
(20, 30, 40)
```

Explanation

The slice includes elements from index `1` to `3`.

---

7. Stores Different Data Types

A tuple can store elements of different data types.

Example

```
student = ("Alice", 20, 89.5, True)

print(student)
```

Output

```
('Alice', 20, 89.5, True)
```

Explanation

The tuple contains a string, integer, float, and boolean.

---

8. Can Contain Nested Tuples

A tuple can contain other tuples.

Example

```
students = (
    ("Alice", 20),
    ("Bob", 21),
```

```
    ("Charlie", 22)
)

print(students)
```

Output

```
(('Alice', 20), ('Bob', 21), ('Charlie', 22))
```

Explanation

Each element of the main tuple is another tuple.

---

9. Memory Efficient

Tuples generally use less memory than lists because they are immutable.

This makes tuples suitable for storing fixed collections of data.

> **Note:** If the data does not need to change, using a tuple is often more memory-efficient than using a list.

---

10. Faster Than Lists

Since tuples are immutable, Python can optimize them internally.

As a result, tuple operations are generally slightly faster than similar list operations.

Summary Table

Characteristic	Tuple
Ordered	✅ Yes
Mutable	❌ No (Immutable)
Allows Duplicates	✅ Yes
Indexed	✅ Yes
Negative Indexing	✅ Yes
Supports Slicing	✅ Yes
Mixed Data Types	✅ Yes
Nested Tuples	✅ Yes
Memory Efficient	✅ Yes
Faster Than Lists	✅ Yes

Key Points

- Tuples preserve the order of elements.
- Tuples are immutable after creation.
- Duplicate values are allowed.

- Tuples support indexing, negative indexing, and slicing.
- Tuples can store different data types and nested tuples.
- Tuples are more memory-efficient and slightly faster than lists.

Advantages of Tuples

Tuples provide several advantages over other data structures, especially when working with **fixed or constant data**.

They are commonly used in Python programs because they are **simple, efficient, and reliable**.

---

1. Immutable

Once a tuple is created, its elements cannot be changed.

This prevents accidental modification of important data.

Example

```
days = ("Monday", "Tuesday", "Wednesday")
```
The values remain unchanged throughout the program.

---

2. Faster Than Lists

Tuples are generally **slightly faster** than lists because they are immutable.

Python performs certain operations more efficiently on tuples.

> **Note:** If your data does not need to change, using a tuple can improve performance.

---

3. More Memory Efficient

Tuples consume **less memory** than lists because Python does not need to allocate extra space for modifications.

This makes tuples a good choice when storing large amounts of fixed data.

---

4. Safer for Constant Data

Tuples are ideal for storing values that should remain constant.

Examples include:

- Days of the week
- Months of the year
- Mathematical constants
- RGB color values
- GPS coordinates

Example

```
days = (
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday"
)
```

---

5. Can Be Used as Dictionary Keys

Since tuples are immutable, they can be used as **dictionary keys**.

Lists cannot be used as dictionary keys because they are mutable.

Example

```
locations = {
    (28.6139, 77.2090): "New Delhi",
    (19.0760, 72.8777): "Mumbai"
}

print(locations[(28.6139, 77.2090)])
```

Output

```
New Delhi
```

---

6. Useful for Returning Multiple Values

Functions can return multiple values using tuples.

Example

```
def student():
    return "Alice", 20

name, age = student()

print(name)
print(age)
```

Output

```
Alice
20
```

---

7. Supports Packing and Unpacking

Tuples make it easy to group and extract multiple values.

Example

```
student = ("Alice", 20, "CSE")

name, age, course = student

print(name)
print(age)
print(course)
```

Output

```
Alice
20
CSE
```

---

8. Maintains Element Order

Tuples preserve the order of elements.

The elements are retrieved in the same order they were inserted.

Example

```
colors = ("Red", "Green", "Blue")

print(colors)
```

Output

```
('Red', 'Green', 'Blue')
```

---

9. Allows Duplicate Values

Tuples can store duplicate elements whenever needed.

Example

```
numbers = (10, 20, 10, 30)

print(numbers)
```

Output

```
(10, 20, 10, 30)
```

---

10. Supports Indexing and Slicing

Tuples support:

- Positive indexing
- Negative indexing
- Slicing

This makes accessing elements simple and efficient.

Example

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
```

Output

```
(20, 30, 40)
```

Summary Table

Advantage	Benefit
Immutable	Prevents accidental modification
Faster	Slightly faster than lists
Memory Efficient	Uses less memory
Safe for Constant Data	Best for fixed information
Dictionary Keys	Can be used as keys
Multiple Return Values	Functions can return multiple values
Packing & Unpacking	Easy assignment of multiple values
Ordered	Maintains insertion order
Allows Duplicates	Duplicate values are supported
Indexing & Slicing	Easy access to elements

Key Points

- Tuples are immutable, making them safer for fixed data.
- Tuples generally use less memory than lists.
- Tuples are slightly faster than lists.
- Tuples can be used as dictionary keys.
- Functions commonly use tuples to return multiple values.
- Tuples support indexing, slicing, packing, and unpacking.
- Tuples are ideal for storing constant or read-only data.

Tuple vs List

Both **tuples** and **lists** are used to store multiple values in Python. They share many similarities, such as maintaining the order of elements and supporting indexing and slicing.

However, they differ in terms of **mutability**, **performance**, **memory usage**, and **use cases**.

> **Note:** Choose a **list** when the data needs to change, and choose a **tuple** when the data should remain constant.

Comparison Table

Feature	Tuple	List
Syntax	`( )`	`[ ]`
Mutable	❌ No (Immutable)	✅ Yes (Mutable)
Ordered	✅ Yes	✅ Yes
Allows Duplicates	✅ Yes	✅ Yes
Indexed	✅ Yes	✅ Yes
Supports Slicing	✅ Yes	✅ Yes
Mixed Data Types	✅ Yes	✅ Yes
Memory Usage	Less	More
Performance	Slightly Faster	Slightly Slower
Built-in Methods	`count()`, `index()`	Many methods (`append()`, `insert()`, `remove()`, etc.)
Dictionary Key	✅ Can be used (if all elements are immutable)	❌ Cannot be used
Best Use	Fixed data	Frequently changing data

Syntax Difference

Tuple

```
student = ("Alice", 20, "CSE")
```

List

```
student = ["Alice", 20, "CSE"]
```

Mutability

Tuple

```
numbers = (10, 20, 30)

numbers[0] = 100
```

Result

```
TypeError: 'tuple' object does not support item assignment
```

List

```
numbers = [10, 20, 30]

numbers[0] = 100

print(numbers)
```

Output

```
[100, 20, 30]
```

Explanation

- Tuple elements cannot be modified.
- List elements can be modified at any time.

Memory Usage

Tuples generally use less memory than lists because they are immutable.

Example

```
student_tuple = ("Alice", 20, "CSE")
student_list = ["Alice", 20, "CSE"]
```

> **Note:** When storing large amounts of constant data, tuples are usually a better choice because they are more memory-efficient.

Performance

Since tuples are immutable, Python can optimize them internally.

As a result:

- Tuples are generally slightly faster.
- Lists are slightly slower because they support modification.

Available Methods

Tuple Methods

```
count()
index()
```

List Methods

```
append()
insert()
extend()
remove()
pop()
clear()
sort()
reverse()
copy()
index()
count()
```

Explanation

Lists provide many methods for modifying data, while tuples provide only methods for searching.

Dictionary Keys

A tuple can be used as a dictionary key because it is immutable.

Example

```
locations = {
    (28.6139, 77.2090): "New Delhi"
}

print(locations[(28.6139, 77.2090)])
```

Output

```
New Delhi
```

Lists cannot be used as dictionary keys because they are mutable.

When Should You Use a Tuple?

Use a tuple when:

- The data should not change.
- You want better memory efficiency.
- You want slightly better performance.
- You need to use the collection as a dictionary key.
- You want to return multiple values from a function.

When Should You Use a List?

Use a list when:

- Elements need to be added or removed.
- Data changes frequently.
- You need list methods like `append()`, `remove()`, or `sort()`.
- You are working with dynamic collections.

Real-Life Examples

Tuple

```
date_of_birth = (15, "August", 2005)
```

A date of birth is fixed and should not change.

List

```
shopping_list = ["Milk", "Bread", "Eggs"]
```

A shopping list changes frequently as items are added or removed.

Summary

Use Tuple When...	Use List When...
Data is fixed	Data changes frequently
Better performance is needed	Frequent modifications are required
Better memory efficiency is needed	More built-in methods are required
Using as dictionary keys	Managing dynamic collections

Key Points

- Both tuples and lists are ordered collections.
- Tuples are immutable, while lists are mutable.
- Tuples use less memory and are slightly faster than lists.
- Lists provide many methods for adding, removing, and modifying elements.
- Tuples are suitable for fixed data, while lists are suitable for dynamic data.
- Choose the data structure based on how your data will be used.

When Should You Use a Tuple?

A tuple is the best choice when your data **should remain unchanged** throughout the execution of a program.

Since tuples are **immutable**, they provide better data protection, slightly better performance, and improved memory efficiency compared to lists.

> **Rule of Thumb:**
> **If the data is fixed, use a tuple. If the data changes frequently, use a list.**

Use a Tuple When...

1. The Data Should Not Change

If the values are constant, a tuple prevents accidental modification.

Example

```
days = (
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday"
)

print(days)
```

---

2. You Want Better Memory Efficiency

Tuples generally use less memory than lists because they are immutable.

This makes them suitable for storing large collections of constant data.

---

3. You Want Better Performance

Tuple operations are generally slightly faster than list operations.

This can improve performance when working with read-only data.

---

4. You Need Dictionary Keys

Only immutable objects can be used as dictionary keys.

Since tuples are immutable, they can be used as keys.

Example

```
locations = {
    (28.6139, 77.2090): "New Delhi"
}

print(locations)
```

---

5. Returning Multiple Values from a Function

Functions often return multiple values as a tuple.

Example

```
def student():
    return "Alice", 20, "CSE"

name, age, course = student()

print(name)
print(age)
print(course)
```

Output

```
Alice
20
CSE
```

---

6. Packing Related Data Together

A tuple can group related values into a single object.

Example

```
student = ("Alice", 101, "Computer Science")
```

The tuple stores:

- Student Name

- Student ID
- Department

Real-World Examples

GPS Coordinates

```
location = (28.6139, 77.2090)
```

Coordinates usually remain constant.

RGB Color

```
red = (255, 0, 0)
```

The RGB values for a color do not change.

Date of Birth

```
date_of_birth = (15, "August", 2005)
```

A person's date of birth is fixed.

Summary Table

Situation	Recommended
Constant Data	✅ Tuple
Frequently Changing Data	❌ Use List
Better Performance	✅ Tuple
Better Memory Efficiency	✅ Tuple
Dictionary Keys	✅ Tuple
Return Multiple Values	✅ Tuple

Key Points

- Use tuples for fixed or read-only data.
- Tuples are ideal for coordinates, dates, colors, and configuration values.
- Tuples provide better memory efficiency and slightly better performance.
- Tuples can be used as dictionary keys.
- If data needs frequent modification, use a list instead.

Creating Tuples

Python provides several ways to create a tuple.

A tuple is usually created using **parentheses** `()`, but Python also creates a tuple when values are separated by commas.

> **Note:** Tuples are immutable, so once created, their elements cannot be modified.

---

1. Creating an Empty Tuple

An empty tuple contains no elements.

Syntax

```
tuple_name = ()
```

Example

```
numbers = ()

print(numbers)
```

Output

```
()
```

---

2. Creating a Single-Element Tuple

A single-element tuple **must include a trailing comma (`,`)**.

Without the comma, Python treats it as a normal value.

Syntax

```
tuple_name = (value,)
```

Example

```
numbers = (10,)

print(numbers)
print(type(numbers))
```

Output

```
(10,)
<class 'tuple'>
```

Incorrect Example

```
numbers = (10)

print(type(numbers))
```

Output

```
<class 'int'>
```

Explanation

`(10)` is treated as an integer, not a tuple.

---

3. Creating a Tuple with Multiple Elements

Multiple elements are separated by commas.

Example

```
fruits = ("Apple", "Banana", "Mango")

print(fruits)
```

Output

```
('Apple', 'Banana', 'Mango')
```

---

4. Creating a Tuple with Mixed Data Types

A tuple can store different data types.

Example

```
student = ("Alice", 20, 89.5, True)

print(student)
```

Output

```
('Alice', 20, 89.5, True)
```

---

5. Creating a Nested Tuple

A tuple can contain other tuples.

Example

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

print(students)
```

Output

```
(('Alice', 20), ('Bob', 21), ('Charlie', 22))
```

---

6. Creating a Tuple Without Parentheses

Python automatically creates a tuple when values are separated by commas.

Example

```
colors = "Red", "Green", "Blue"

print(colors)
```

Output

```
('Red', 'Green', 'Blue')
```

---

7. Creating a Tuple Using the `tuple()` Constructor ⭐

Python provides the built-in `tuple()` function to create a tuple from another iterable.

Example 1: From a List

```
numbers = tuple([10, 20, 30])

print(numbers)
```

Output

```
(10, 20, 30)
```

Example 2: From a String

```
letters = tuple("Python")

print(letters)
```

Output

```
('P', 'y', 't', 'h', 'o', 'n')
```

Example 3: From a Range

```
numbers = tuple(range(1, 6))

print(numbers)
```

Output

```
(1, 2, 3, 4, 5)
```

Summary Table

Method	Example
Empty Tuple	`()`
Single Element	`(10,)`
Multiple Elements	`(1, 2, 3)`
Mixed Data Types	`("Alice", 20, True)`
Nested Tuple	`((1, 2), (3, 4))`
Without Parentheses	`1, 2, 3`
`tuple()` Constructor	`tuple([1, 2, 3])`

Key Points

- Tuples are usually created using parentheses `()`.
- A single-element tuple must end with a comma.
- Tuples can store different data types.
- Tuples can contain other tuples.
- Python can create tuples without parentheses when values are separated by commas.
- The `tuple()` constructor converts other iterables into tuples.

Tuple Constructor (`tuple()`)

Python provides the built-in **`tuple()`** constructor to create a tuple from another iterable object.

An **iterable** is any object that can be looped through, such as:

- List
- String
- Set
- Range
- Dictionary

> **Note:** If no argument is passed, `tuple()` creates an empty tuple.

Syntax

```
tuple(iterable)
```

Example 1: Creating an Empty Tuple

```
numbers = tuple()

print(numbers)
```

Output

```
()
```

Explanation

Since no iterable is provided, Python creates an empty tuple.

Example 2: Creating a Tuple from a List

```
numbers = [10, 20, 30, 40]

result = tuple(numbers)

print(result)
```

Output

```
(10, 20, 30, 40)
```

Explanation

The list is converted into a tuple.

Example 3: Creating a Tuple from a String

```
text = "Python"

letters = tuple(text)

print(letters)
```

Output

```
('P', 'y', 't', 'h', 'o', 'n')
```

Explanation

Each character becomes an element of the tuple.

Example 4: Creating a Tuple from a Set

```
numbers = {10, 20, 30}

result = tuple(numbers)

print(result)
```

Possible Output

```
(10, 20, 30)
```

> **Note:** Sets are unordered, so the order of elements may vary.

Example 5: Creating a Tuple from `range()`

```
numbers = tuple(range(1, 6))

print(numbers)
```

Output

```
(1, 2, 3, 4, 5)
```

Explanation

The values generated by `range()` are converted into a tuple.

Example 6: Creating a Tuple from a Dictionary

```
student = {
    "name": "Alice",
    "age": 20,
    "course": "CSE"
}

result = tuple(student)

print(result)
```

Output

```
('name', 'age', 'course')
```

Explanation

When a dictionary is passed to `tuple()`, only its **keys** are converted into tuple elements.

Summary Table

Iterable	Example	Result
List	`tuple([1, 2, 3])`	`(1, 2, 3)`
String	`tuple("Python")`	`('P', 'y', 't', 'h', 'o', 'n')`
Set	`tuple({1, 2, 3})`	`(1, 2, 3)` *(order may vary)*
Range	`tuple(range(5))`	`(0, 1, 2, 3, 4)`
Dictionary	`tuple({"a":1, "b":2})`	`('a', 'b')`

Key Points

- `tuple()` is a built-in constructor used to create tuples.
- It converts iterable objects into tuples.
- Calling `tuple()` without arguments creates an empty tuple.
- When used with a dictionary, only the dictionary keys are included.

Accessing Tuple Elements

Each element in a tuple has a unique **index**.

You can access an element by using its index inside square brackets `[]`.

Python uses **zero-based indexing**, which means the first element has an index of `0`.

> **Note:** Tuples are immutable, but their elements can still be accessed using indexes.

Syntax

```
tuple_name[index]
```

Example 1: Accessing the First Element

```
fruits = ("Apple", "Banana", "Mango")

print(fruits[0])
```

Output

```
Apple
```

Explanation

The first element is located at index `0`.

Example 2: Accessing the Second Element

```
fruits = ("Apple", "Banana", "Mango")

print(fruits[1])
```

Output

```
Banana
```

Example 3: Accessing the Last Element

```
fruits = ("Apple", "Banana", "Mango")

print(fruits[2])
```

Output

```
Mango
```

Tuple Index Representation

```
fruits = ("Apple", "Banana", "Mango", "Orange")
```

Element	Apple	Banana	Mango	Orange
Positive Index	0	1	2	3
Negative Index	-4	-3	-2	-1

Accessing Nested Tuple Elements

Nested tuples require multiple indexes.

Example

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

print(students[1][0])
```

Output

```
Bob
```

Explanation

- `students[1]` selects the second tuple.
- `[0]` selects the first element of that tuple.

Accessing Multiple Elements

```
student = ("Alice", 20, "CSE")
```

```
print(student[0])
print(student[2])
```

Output

```
Alice
CSE
```

Invalid Index

Accessing an index that does not exist raises an **IndexError**.

Example

```
numbers = (10, 20, 30)

print(numbers[5])
```

Output

```
IndexError: tuple index out of range
```

Summary Table

Operation	Example
First Element	`tuple[0]`
Second Element	`tuple[1]`
Last Element	`tuple[-1]`
Nested Tuple	`tuple[1][0]`

Key Points

- Tuple indexing starts from `0`.
- Use square brackets `[]` to access elements.
- Positive indexing starts from the beginning.
- Negative indexing starts from the end.
- Nested tuples require multiple indexes.
- Accessing an invalid index raises an `IndexError`.

Indexing

Indexing is used to access individual elements of a tuple.

Python uses **zero-based indexing**, which means the **first element** has an index of **0**, the **second element** has an index of **1**, and so on.

> **Note:** Indexing allows you to access tuple elements, but since tuples are immutable, you cannot modify them using indexes.

Syntax

```
tuple_name[index]
```

Example 1: Accessing the First Element

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(fruits[0])
```

Output

```
Apple
```

Explanation

The first element `"Apple"` is located at index `0`.

Example 2: Accessing the Second Element

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(fruits[1])
```

Output

```
Banana
```

Example 3: Accessing the Last Element Using Positive Index

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(fruits[3])
```

Output

```
Orange
```

Positive Index Representation

```
fruits = ("Apple", "Banana", "Mango", "Orange")
```

Element	Apple	Banana	Mango	Orange
Index	0	1	2	3

Accessing Elements from a Nested Tuple

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)
```

```
print(students[1][0])
```
Output

Bob
Explanation

- `students[1]` selects the second tuple.
- `[0]` selects the first element of that tuple.

Invalid Index

Trying to access an index that does not exist raises an **IndexError**.

Example

```
numbers = (10, 20, 30)

print(numbers[5])
```
Output

```
IndexError: tuple index out of range
```

Key Points

- Indexing starts from `0`.
- Use square brackets `[]` to access tuple elements.
- Positive indexes count from left to right.
- Nested tuples require multiple indexes.
- Accessing an invalid index raises an `IndexError`.

Negative Indexing

Python also supports **negative indexing**, which allows you to access elements from the **end** of a tuple.

Negative indexing starts with `-1`, which represents the **last element**.

> **Note:** Negative indexing is useful when you want to access elements from the end without knowing the total number of elements.

Syntax

```
tuple_name[-index]
```

Example 1: Accessing the Last Element

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(fruits[-1])
```
Output

Orange

Explanation

`-1` always refers to the last element.

Example 2: Accessing the Second Last Element

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(fruits[-2])
```

Output

Mango

Example 3: Accessing the First Element

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(fruits[-4])
```

Output

Apple

Negative Index Representation

```
fruits = ("Apple", "Banana", "Mango", "Orange")
```

Element	Apple	Banana	Mango	Orange
Negative Index	-4	-3	-2	-1

Accessing Elements in a Nested Tuple

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

print(students[-1][-1])
```

Output

22

Explanation

- `students[-1]` selects the last tuple.
- `[-1]` selects the last element of that tuple.

Invalid Negative Index

Accessing a negative index outside the tuple range raises an **IndexError**.

Example

```
numbers = (10, 20, 30)

print(numbers[-5])
```

Output

```
IndexError: tuple index out of range
```

Positive vs Negative Indexing

Positive Index	Negative Index	Element
0	-4	Apple
1	-3	Banana
2	-2	Mango
3	-1	Orange

Key Points

- Negative indexing starts from `-1`.
- `-1` represents the last element of the tuple.
- Negative indexing makes it easy to access elements from the end.
- It works with nested tuples as well.
- Accessing an invalid negative index raises an `IndexError`.

Tuple Slicing

Slicing is used to access a portion of a tuple by specifying a range of indexes.

Instead of accessing a single element, slicing returns a **new tuple** containing the selected elements.

> **Note:** Slicing does not modify the original tuple because tuples are immutable.

Syntax

```
tuple_name[start : stop : step]
```

Syntax Explanation

- **start** → Starting index (included).
- **stop** → Ending index (excluded).
- **step** → Number of positions to move (optional).

Example 1: Basic Slicing

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
```

Output

```
(20, 30, 40)
```

Explanation

The slice starts at index `1` and ends before index `4`.

Example 2: Omitting the Start Index

```
numbers = (10, 20, 30, 40, 50)

print(numbers[:3])
```

Output

```
(10, 20, 30)
```

Example 3: Omitting the Stop Index

```
numbers = (10, 20, 30, 40, 50)

print(numbers[2:])
```

Output

```
(30, 40, 50)
```

Example 4: Copying an Entire Tuple

```
numbers = (10, 20, 30, 40, 50)

print(numbers[:])
```

Output

```
(10, 20, 30, 40, 50)
```

Example 5: Using the Step Value

```
numbers = (10, 20, 30, 40, 50, 60)

print(numbers[::2])
```

Output

```
(10, 30, 50)
```

Explanation

Every second element is selected.

Example 6: Reversing a Tuple

```
numbers = (10, 20, 30, 40, 50)

print(numbers[::-1])
```

Output

```
(50, 40, 30, 20, 10)
```

Explanation

A step value of `-1` returns the tuple in reverse order.

Example 7: Negative Index Slicing

```
numbers = (10, 20, 30, 40, 50)

print(numbers[-4:-1])
```

Output

```
(20, 30, 40)
```

Slicing Examples

Expression	Result
`t[1:4]`	Elements from index 1 to 3
`t[:3]`	First three elements
`t[2:]`	Elements from index 2 to the end
`t[:]`	Entire tuple
`t[::2]`	Every second element
`t[::-1]`	Reverse the tuple

Key Points

- Slicing returns a new tuple.
- The original tuple remains unchanged.
- The `start` index is included.
- The `stop` index is excluded.
- The `step` value controls how elements are selected.
- A negative step can be used to reverse a tuple.

Updating Tuples

Unlike lists, **tuples are immutable**, which means their elements **cannot be changed, added, or removed** after the tuple has been created.

If you try to modify a tuple directly, Python raises a **TypeError**.

> **Note:** Although tuples cannot be updated directly, there are indirect methods to create a modified version of a tuple, which will be covered in the next section.

Tuple Immutability

One of the most important characteristics of a tuple is that it is **immutable**.

Immutability means that once a tuple is created, its elements **cannot be changed, added, or removed**.

In other words, the size and contents of a tuple remain fixed throughout the program.

> **Note:** Although the tuple itself cannot be modified, it can still be accessed, iterated, sliced, and used in expressions.

What Does Immutable Mean?

An immutable object cannot be modified after it has been created.

For tuples, this means you **cannot**:

- Change an existing element.
- Add a new element.
- Remove an existing element.

Example 1: Trying to Modify an Element

```
numbers = (10, 20, 30)

numbers[0] = 100
```

Output

```
TypeError: 'tuple' object does not support item assignment
```

Explanation

Python raises a `TypeError` because tuple elements cannot be changed.

Example 2: Trying to Add an Element

```
numbers = (10, 20, 30)

numbers.append(40)
```

Output

```
AttributeError: 'tuple' object has no attribute 'append'
```

Explanation

Unlike lists, tuples do not have an `append()` method.

Example 3: Trying to Remove an Element

```
numbers = (10, 20, 30)
```

```
numbers.remove(20)
```

Output

```
AttributeError: 'tuple' object has no attribute 'remove'
```

Explanation

Tuples do not support removing elements.

What Operations Are Allowed?

Although tuples cannot be modified, many operations are still allowed.

Example

```
fruits = ("Apple", "Banana", "Mango")

print(fruits[0])
print(len(fruits))
print("Mango" in fruits)
print(fruits[1:])
```

Output

```
Apple
3
True
('Banana', 'Mango')
```

Explanation

You can:

- Access elements
- Find the length
- Search for elements
- Slice the tuple

These operations do not modify the original tuple.

Why Are Tuples Immutable?

Python makes tuples immutable for several reasons:

1. Data Protection

Important data cannot be changed accidentally.

2. Better Performance

Tuple operations are generally slightly faster than list operations.

3. Memory Efficiency

Tuples use less memory than lists because they are immutable.

4. Dictionary Keys

Tuples can be used as dictionary keys, while lists cannot.

Real-Life Example

```
months = (
    "January", "February", "March",
    "April", "May", "June",
    "July", "August", "September",
    "October", "November", "December"
)

print(months)
```

The names of the months remain constant, making a tuple an appropriate choice.

Mutable vs Immutable

Mutable	Immutable
Data can be changed after creation.	Data cannot be changed after creation.
Example: List	Example: Tuple
Supports adding, removing, and updating elements.	Does not support adding, removing, or updating elements.

Summary Table

Operation	Tuple
Read Elements	✅
Update Elements	❌
Add Elements	❌
Remove Elements	❌
Slice Tuple	✅
Iterate Through Tuple	✅
Search Elements	✅

Key Points

- Tuples are immutable.
- Elements cannot be updated, added, or removed after creation.
- Tuples can still be accessed, sliced, and iterated.
- Immutability improves data safety, memory efficiency, and performance.
- Tuples are commonly used for storing fixed or constant data.

Updating Tuples (Indirect Method)

Since tuples are **immutable**, their elements cannot be modified directly.

However, you can update a tuple **indirectly** by converting it into a list, making the required changes, and then converting it back into a tuple.

> **Note:** The original tuple is not modified. A new tuple is created with the updated values.

Steps to Update a Tuple

1. Convert the tuple into a list using `list()`.
2. Modify the list.
3. Convert the list back into a tuple using `tuple()`.

Updating an Element

Example

```
fruits = ("Apple", "Banana", "Mango")

temp = list(fruits)

temp[1] = "Orange"

fruits = tuple(temp)

print(fruits)
```

Output

```
('Apple', 'Orange', 'Mango')
```

Adding an Element

Example

```
numbers = (10, 20, 30)

temp = list(numbers)

temp.append(40)

numbers = tuple(temp)

print(numbers)
```

Output

```
(10, 20, 30, 40)
```

Removing an Element

Example

```
numbers = (10, 20, 30, 40)

temp = list(numbers)

temp.remove(20)

numbers = tuple(temp)

print(numbers)
```

Output

```
(10, 30, 40)
```

Creating a New Tuple Using Concatenation

You can also create a new tuple by joining tuples.

Example

```
numbers = (10, 20, 30)

numbers = numbers + (40,)

print(numbers)
```

Output

```
(10, 20, 30, 40)
```

Summary Table

Operation	Indirect Method
Update an Element	Convert → Modify → Convert Back
Add an Element	Convert → `append()` → Convert Back
Remove an Element	Convert → `remove()` → Convert Back
Join Tuples	Use `+` operator

Key Points

- Tuples cannot be modified directly.
- Convert the tuple into a list to make changes.
- Convert the list back into a tuple after modification.
- Concatenation (`+`) creates a new tuple instead of modifying the existing one.

Tuple Packing & Unpacking

Python provides a simple way to group multiple values into a tuple and later extract those values into individual variables.

This process is known as **Tuple Packing** and **Tuple Unpacking**.

Tuple Packing

Tuple Packing is the process of storing multiple values together into a single tuple.

When you separate multiple values using commas (`,`), Python automatically creates a tuple. This process is called **packing**.

> **Note:** Parentheses are optional while packing a tuple. The comma (`,`) is what actually creates the tuple.

Syntax

Without Parentheses

```
tuple_name = value1, value2, value3
```

With Parentheses

```
tuple_name = (value1, value2, value3)
```

Both syntaxes create exactly the same tuple.

Example 1: Packing Without Parentheses

```
student = "Alice", 20, "CSE"

print(student)
print(type(student))
```

Output

```
('Alice', 20, 'CSE')
<class 'tuple'>
```

Explanation

Python automatically packs the three values into a tuple.

Example 2: Packing With Parentheses

```
student = ("Alice", 20, "CSE")

print(student)
print(type(student))
```

Output

```
('Alice', 20, 'CSE')
<class 'tuple'>
```

Example 3: Packing Different Data Types

```
data = ("Python", 3.14, True, 100)

print(data)
```

Output

```
('Python', 3.14, True, 100)
```

Explanation

A tuple can store elements of different data types.

Example 4: Packing a Single Element

```
number = (10,)

print(number)
print(type(number))
```

Output

```
(10,)
<class 'tuple'>
```

Explanation

A comma is required when packing a single element. Without the comma, Python treats it as a normal integer.

Example 5: Packing Function Return Values

```
def student():
    return "Alice", 20, "CSE"

data = student()

print(data)
```

Output

```
('Alice', 20, 'CSE')
```

Explanation

The function returns multiple values, and Python automatically packs them into a tuple.

Example 6: Packing Multiple Variables

```
name = "Alice"
age = 20
course = "CSE"

student = name, age, course

print(student)
```

Output

```
('Alice', 20, 'CSE')
```

Real-World Example

Suppose you want to store a student's information.

```
student = (
    "Rahul",
    101,
    "Computer Science",
    8.75
)

print(student)
```

Output

```
('Rahul', 101, 'Computer Science', 8.75)
```

Instead of creating separate variables, tuple packing stores all related information in a single tuple.

Advantages of Tuple Packing

- Groups multiple values into one object.
- Makes the code cleaner and more organized.
- Useful for returning multiple values from functions.
- Requires less memory than lists.
- Makes data easier to pass between functions.

Summary Table

Feature	Description
Purpose	Store multiple values in a single tuple
Parentheses Required	No
Comma Required	Yes
Supports Mixed Data Types	✅
Used in Function Returns	✅

Key Points

- Tuple packing combines multiple values into one tuple.
- Python automatically creates a tuple when values are separated by commas.
- Parentheses are optional during packing.
- A single-element tuple must include a trailing comma.
- Tuple packing is commonly used when returning multiple values from functions.

Tuple Unpacking

Tuple Unpacking is the process of assigning the elements of a tuple to individual variables.

Instead of accessing each element using indexes, Python automatically assigns each element to a corresponding variable.

> **Note:** The number of variables must exactly match the number of elements in the tuple unless you use **extended unpacking (`*`)**, which will be covered in the next section.

Syntax

```
variable1, variable2, variable3 = tuple_name
```

Example 1: Basic Tuple Unpacking

```
student = ("Alice", 20, "CSE")

name, age, course = student

print(name)
print(age)
print(course)
```

Output

```
Alice
20
CSE
```

Explanation

Python assigns:

- `"Alice"` → `name`
- `20` → `age`
- `"CSE"` → `course`

Example 2: Unpacking Numbers

```
numbers = (10, 20, 30)

a, b, c = numbers

print(a)
print(b)
print(c)
```

Output

```
10
20
30
```

Example 3: Unpacking Different Data Types

```
data = ("Python", 3.14, True)

language, version, available = data
```

```
print(language)
print(version)
print(available)
```

Output

```
Python
3.14
True
```

Example 4: Using Unpacked Variables

```
dimensions = (15, 8)

length, width = dimensions

area = length * width

print(area)
```

Output

```
120
```

Explanation

The unpacked variables can be used like normal variables.

Example 5: Swapping Variables

Tuple unpacking makes swapping variables simple.

```
a = 10
b = 20

a, b = b, a

print(a)
print(b)
```

Output

```
20
10
```

Explanation

Python packs the values on the right into a tuple and then unpacks them into the variables on the left.

Example 6: Unpacking Function Return Values

```
def student():
    return "Alice", 20, "CSE"

name, age, course = student()

print(name)
print(age)
print(course)
```

Output

```
Alice
20
CSE
```

Explanation

The function returns a tuple, which is immediately unpacked into three variables.

Incorrect Unpacking

The number of variables must match the number of tuple elements.

Example

```
student = ("Alice", 20, "CSE")

name, age = student
```

Output

```
ValueError: too many values to unpack (expected 2)
```

Another Example

```
student = ("Alice", 20)

name, age, course = student
```

Output

```
ValueError: not enough values to unpack (expected 3, got 2)
```

Real-World Example

Suppose a function returns a student's details.

```
def get_student():
    return "Rahul", 101, 8.9

name, roll_no, cgpa = get_student()

print(name)
print(roll_no)
print(cgpa)
```

Output

```
Rahul
101
8.9
```

Using tuple unpacking makes the code clean and easy to understand.

Advantages of Tuple Unpacking

- Makes code cleaner and more readable.

- Eliminates the need for indexing.
- Useful when functions return multiple values.
- Makes variable swapping simple.
- Reduces the number of assignment statements.

Summary Table

Feature	Description
Purpose	Assign tuple elements to variables
Number of Variables	Must match the number of tuple elements
Supports Mixed Data Types	✅
Used in Function Returns	✅
Useful for Variable Swapping	✅

Key Points

- Tuple unpacking assigns tuple elements to individual variables.
- Each variable receives one value from the tuple.
- The number of variables should match the number of tuple elements.
- Tuple unpacking is commonly used with function return values.
- Python also uses tuple unpacking for swapping variables.
- Extended unpacking using the `*` operator is covered in the next section.

Extended Unpacking (`*`)

Sometimes, you may not know the exact number of values in a tuple or you may want to collect multiple elements into a single variable.

Python provides **Extended Unpacking** using the **asterisk (`*`) operator**.

The `*` operator collects the remaining elements into a **list**.

> **Note:** Only one variable can be prefixed with `*` in a single unpacking statement.

Syntax

```
first, *middle, last = tuple_name
```

Example 1: Collecting Middle Elements

```
numbers = (10, 20, 30, 40, 50)

first, *middle, last = numbers

print(first)
```

```
print(middle)
print(last)
```

Output

```
10
[20, 30, 40]
50
```

Explanation

- `first` gets the first element.
- `last` gets the last element.
- `middle` collects all remaining elements as a list.

Example 2: Collecting Remaining Elements

```
fruits = ("Apple", "Banana", "Mango", "Orange")

first, *remaining = fruits

print(first)
print(remaining)
```

Output

```
Apple
['Banana', 'Mango', 'Orange']
```

Example 3: Collecting Initial Elements

```
fruits = ("Apple", "Banana", "Mango", "Orange")

*beginning, last = fruits

print(beginning)
print(last)
```

Output

```
['Apple', 'Banana', 'Mango']
Orange
```

Example 4: Ignoring Some Values

```
student = ("Alice", 20, "CSE", "India")

name, *_, country = student

print(name)
print(country)
```

Output

```
Alice
India
```

Explanation

The variable `_` is commonly used to indicate values that are intentionally ignored.

Example 5: Extended Unpacking with Function Return

```
def numbers():
    return (10, 20, 30, 40, 50)

first, *others = numbers()

print(first)
print(others)
```

Output

```
10
[20, 30, 40, 50]
```

Rules of Extended Unpacking

- Only one variable can use the `*` operator.
- The starred variable always stores a **list**.
- The `*` variable can appear at the beginning, middle, or end of the assignment.

Invalid Example

```
numbers = (10, 20, 30, 40)

*a, *b = numbers
```

Output

```
SyntaxError: multiple starred expressions in assignment
```

Summary Table

Expression	Result
`a, *b = t`	First element in `a`, remaining elements in `b`
`*a, b = t`	Last element in `b`, remaining elements in `a`
`a, *b, c = t`	First in `a`, last in `c`, middle elements in `b`

Key Points

- Extended unpacking uses the `*` operator.
- The starred variable collects multiple elements into a list.
- Only one starred variable is allowed in an unpacking statement.
- The starred variable can be placed at the beginning, middle, or end.
- Extended unpacking is useful when the number of elements is unknown.

Looping Through Tuples

A **loop** is used to access each element of a tuple one by one.

Since tuples are iterable, they can be traversed using different types of loops.

The most common loops used with tuples are:

- `for` loop
- `while` loop
- `for` loop with `range()`
- `enumerate()` function

Using `for` Loop

The **`for` loop** is the simplest and most commonly used way to iterate through the elements of a tuple.

It automatically visits each element one by one without requiring indexes.

> **Note:** Since tuples are iterable objects, they can be directly used with a `for` loop.

Syntax

In [ ]:
```
for variable in tuple_name:
    print(variable)
```

Example 1: Print All Elements

In [ ]:
```
fruits = ("Apple", "Banana", "Mango", "Orange")

for fruit in fruits:
    print(fruit)
```

Output

```
Apple
Banana
Mango
Orange
```

Explanation

The `for` loop visits each element of the tuple and stores it in the variable `fruit` one at a time.

Example 2: Loop Through Numbers

In [ ]:
```
numbers = (10, 20, 30, 40, 50)

for num in numbers:
    print(num)
```

Output

```
10
20
```

```
30
40
50
```

Example 3: Find the Sum of Tuple Elements

```
numbers = (10, 20, 30, 40)

total = 0

for num in numbers:
    total += num

print("Sum =", total)
```

Output

```
Sum = 100
```

Explanation

The loop adds each number to the variable `total`.

Example 4: Print Squares of Numbers

```
numbers = (1, 2, 3, 4, 5)

for num in numbers:
    print(num ** 2)
```

Output

```
1
4
9
16
25
```

Example 5: Loop Through a Tuple of Strings

```
languages = ("Python", "Java", "C", "JavaScript")

for language in languages:
    print("Programming Language:", language)
```

Output

```
Programming Language: Python
Programming Language: Java
Programming Language: C
Programming Language: JavaScript
```

Example 6: Loop Through a Nested Tuple

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)
```

```
for student in students:
    print(student)
```

Output

```
('Alice', 20)
('Bob', 21)
('Charlie', 22)
```

Example 7: Tuple Unpacking Inside a `for` Loop

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

for name, age in students:
    print(name, "-", age)
```

Output

```
Alice - 20
Bob - 21
Charlie - 22
```

Explanation

Python automatically unpacks each inner tuple into the variables `name` and `age`.

Flow of a `for` Loop

For the tuple:

```
fruits = ("Apple", "Banana", "Mango")
```

The loop executes as follows:

Iteration	Variable (`fruit`)	Printed Value
1	Apple	Apple
2	Banana	Banana
3	Mango	Mango

Advantages of Using a `for` Loop

- Simple and easy to understand.
- No need to manage indexes manually.
- Suitable for iterating through all tuple elements.
- Works with tuples of any data type.
- Can directly unpack nested tuples.

Real-World Example

Suppose you want to display the names of students.

```
students = (
    "Rahul",
    "Priya",
    "Aman",
    "Neha"
)

for student in students:
    print("Student:", student)
```

Output

```
Student: Rahul
Student: Priya
Student: Aman
Student: Neha
```

Summary Table

Feature	Description
Loop Type	`for` loop
Index Required	❌ No
Accesses	One element at a time
Works with Nested Tuples	✅ Yes
Supports Tuple Unpacking	✅ Yes

Key Points

- The `for` loop is the easiest way to iterate through a tuple.
- It automatically accesses one element at a time.
- No index variable is required.
- It works with numbers, strings, mixed data types, and nested tuples.
- Tuple unpacking can be used directly inside a `for` loop for cleaner code.

Using `range()`

The `range()` function is commonly used with a `for` loop to iterate through a tuple using **index numbers** instead of directly accessing the elements.

This approach is useful when you need both the **index** and the **value** of each element.

> **Note:** Since tuples do not provide indexes automatically in a `for` loop, `range()` is often used together with the `len()` function.

Syntax

```
for i in range(len(tuple_name)):
    print(tuple_name[i])
```

How It Works

- `len(tuple_name)` returns the total number of elements.
- `range()` generates index values from `0` to `len(tuple_name) - 1`.
- Each index is then used to access the corresponding tuple element.

Example 1: Print All Elements

```
fruits = ("Apple", "Banana", "Mango", "Orange")

for i in range(len(fruits)):
    print(fruits[i])
```

Output

```
Apple
Banana
Mango
Orange
```

Explanation

The loop generates indexes:

- `0`
- `1`
- `2`
- `3`

Each index is used to access one element of the tuple.

Example 2: Print Index and Element

```
fruits = ("Apple", "Banana", "Mango", "Orange")

for i in range(len(fruits)):
    print(i, fruits[i])
```

Output

```
0 Apple
1 Banana
2 Mango
3 Orange
```

Example 3: Print Only Even Index Elements

```
numbers = (10, 20, 30, 40, 50, 60)

for i in range(0, len(numbers), 2):
    print(numbers[i])
```

Output

```
10
30
50
```

Explanation

The third argument of `range()` specifies the **step size**, so the loop visits every second index.

Example 4: Print Tuple in Reverse Order

```
numbers = (10, 20, 30, 40, 50)

for i in range(len(numbers) - 1, -1, -1):
    print(numbers[i])
```

Output

```
50
40
30
20
10
```

Explanation

The loop starts from the last index and moves backwards.

Example 5: Calculate the Sum of Elements

```
numbers = (10, 20, 30, 40)

total = 0

for i in range(len(numbers)):
    total += numbers[i]

print("Sum =", total)
```

Output

```
Sum = 100
```

Example 6: Print Square of Each Element

```
numbers = (1, 2, 3, 4, 5)

for i in range(len(numbers)):
    print(numbers[i] ** 2)
```

Output

```
1
4
9
16
25
```

Flow of `range(len())`

For the tuple:

```
fruits = ("Apple", "Banana", "Mango")
```

The loop executes as follows:

Iteration	Index (`i`)	Element (`fruits[i]`)
1	0	Apple
2	1	Banana

Iteration	Index (`i`)	Element (`fruits[i]`)
3	2	Mango

When Should You Use `range()`?

Use `range()` when you need:

- The index of each element.
- To access elements by their position.
- To skip elements using a step value.
- To traverse a tuple in reverse order.
- More control over the iteration process.

`for` Loop vs `range()`

`for` Loop	`range()`
Iterates directly over elements	Iterates over indexes
Simpler syntax	More control over iteration
No index available by default	Index is available
Best for reading values	Best for index-based operations

Real-World Example

Suppose you want to display the roll number along with each student's name.

```
students = ("Rahul", "Priya", "Aman", "Neha")

for i in range(len(students)):
    print("Roll No.", i + 1, ":", students[i])
```

Output

```
Roll No. 1 : Rahul
Roll No. 2 : Priya
Roll No. 3 : Aman
Roll No. 4 : Neha
```

Summary Table

Feature	Description
Loop Type	`for` loop with `range()`
Uses Indexes	✅ Yes
Requires `len()`	✅ Yes
Supports Step Value	✅ Yes
Supports Reverse Traversal	✅ Yes

Key Points

- `range()` is used to iterate through tuple indexes.
- `len()` determines the number of elements in the tuple.
- Use `tuple_name[i]` to access elements by index.
- `range()` provides more control than a simple `for` loop.
- It is useful for reverse traversal, skipping elements, and index-based operations.

Using `while` Loop

A **`while` loop** can also be used to iterate through the elements of a tuple.

Unlike the `for` loop, a `while` loop uses an **index variable** to access each element.

> **Note:** The `while` loop is useful when you need more control over the iteration process, such as changing the index manually or using custom conditions.

Syntax

```
i = 0

while i < len(tuple_name):
    print(tuple_name[i])
    i += 1
```

How It Works

1. Initialize the index variable.
2. Check the loop condition.
3. Access the tuple element using the index.
4. Increase the index.
5. Repeat until the condition becomes `False`.

Example 1: Print All Elements

```
fruits = ("Apple", "Banana", "Mango", "Orange")

i = 0

while i < len(fruits):
    print(fruits[i])
    i += 1
```

Output

```
Apple
Banana
Mango
Orange
```

Explanation

The variable `i` starts at `0` and increases by `1` after each iteration until all elements have been printed.

Example 2: Print Index and Element

```
fruits = ("Apple", "Banana", "Mango", "Orange")

i = 0

while i < len(fruits):
    print(i, fruits[i])
    i += 1
```

Output

```
0 Apple
1 Banana
2 Mango
3 Orange
```

Example 3: Calculate the Sum of Elements

```
numbers = (10, 20, 30, 40)

total = 0
i = 0

while i < len(numbers):
    total += numbers[i]
    i += 1

print("Sum =", total)
```

Output

```
Sum = 100
```

Example 4: Print Elements in Reverse Order

```
numbers = (10, 20, 30, 40, 50)

i = len(numbers) - 1

while i >= 0:
    print(numbers[i])
    i -= 1
```

Output

```
50
40
30
20
10
```

Explanation

The loop starts from the last index and moves backwards until it reaches the first element.

Example 5: Print Only Even Index Elements

```
numbers = (10, 20, 30, 40, 50, 60)

i = 0
```

```
while i < len(numbers):
    print(numbers[i])
    i += 2
```

Output

```
10
30
50
```

Example 6: Print Each Element with a Message

```
languages = ("Python", "Java", "C")

i = 0

while i < len(languages):
    print("Programming Language:", languages[i])
    i += 1
```

Output

```
Programming Language: Python
Programming Language: Java
Programming Language: C
```

Flow of a `while` Loop

For the tuple:

```
fruits = ("Apple", "Banana", "Mango")
```

The loop executes as follows:

Iteration	Value of `i`	Element Printed
1	0	Apple
2	1	Banana
3	2	Mango

When Should You Use a `while` Loop?

Use a `while` loop when you need to:

- Control the index manually.
- Traverse a tuple in reverse order.
- Skip elements by changing the index.
- Stop the loop based on a custom condition.
- Perform index-based operations.

`for` Loop vs `while` Loop

`for` Loop	`while` Loop
Simpler syntax	Requires an index variable
Automatically moves to the next element	Index must be updated manually
Best for iterating over all elements	Best for custom iteration logic
Less chance of infinite loops	Incorrect index updates may cause infinite loops

---

Real-World Example

Suppose you want to display the serial number of each product.

```
products = ("Laptop", "Mouse", "Keyboard", "Monitor")

i = 0

while i < len(products):
    print("Product", i + 1, ":", products[i])
    i += 1
```

Output

```
Product 1 : Laptop
Product 2 : Mouse
Product 3 : Keyboard
Product 4 : Monitor
```

Summary Table

Feature	Description
Loop Type	`while` loop
Uses Indexes	✅ Yes
Requires `len()`	✅ Yes
Index Updated Manually	✅ Yes
Supports Reverse Traversal	✅ Yes

Key Points

- A `while` loop iterates through a tuple using indexes.
- The index variable should be initialized before the loop.
- The index must be updated inside the loop to avoid an infinite loop.
- `while` loops provide more control over the iteration process.
- They are useful for reverse traversal, skipping elements, and custom iteration logic.

Using `enumerate()`

The `enumerate()` function is used to iterate through a tuple while keeping track of both the **index** and the **element** at the same time.

Instead of manually creating an index variable, `enumerate()` automatically provides it during each iteration.

> **Note:** `enumerate()` is the preferred way to access both the index and the value while looping through a tuple.

Syntax

```
for index, value in enumerate(tuple_name):
    print(index, value)
```

How It Works

- `enumerate()` takes an iterable (such as a tuple).
- It returns both the **index** and the **element** for each iteration.
- The index starts from `0` by default.

Example 1: Print Index and Element

```
fruits = ("Apple", "Banana", "Mango", "Orange")

for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Output

```
0 Apple
1 Banana
2 Mango
3 Orange
```

Explanation

During each iteration:

- `index` stores the position.
- `fruit` stores the corresponding element.

Example 2: Custom Starting Index

You can change the starting index using the `start` parameter.

```
fruits = ("Apple", "Banana", "Mango")

for index, fruit in enumerate(fruits, start=1):
    print(index, fruit)
```

Output

```
1 Apple
2 Banana
3 Mango
```

Explanation

Instead of starting from `0`, the index starts from `1`.

Example 3: Display Student Roll Numbers

```
students = ("Rahul", "Priya", "Aman", "Neha")

for roll_no, student in enumerate(students, start=101):
    print(roll_no, student)
```

Output

```
101 Rahul
102 Priya
103 Aman
104 Neha
```

Example 4: Print Index and Square of Numbers

```
numbers = (2, 4, 6, 8)

for index, number in enumerate(numbers):
    print(index, number ** 2)
```

Output

```
0 4
1 16
2 36
3 64
```

Example 5: Find the Position of an Element

```
languages = ("Python", "Java", "C", "JavaScript")

for index, language in enumerate(languages):
    print(language, "is at index", index)
```

Output

```
Python is at index 0
Java is at index 1
C is at index 2
JavaScript is at index 3
```

Example 6: Nested Tuple with `enumerate()`

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

for index, (name, age) in enumerate(students):
    print(index, name, age)
```

Output

```
0 Alice 20
1 Bob 21
2 Charlie 22
```

Explanation

- `enumerate()` provides the index.

- Tuple unpacking extracts the `name` and `age` from each nested tuple.

`enumerate()` vs `range()`

`enumerate()`	`range()`
Returns both index and value	Returns only indexes
Cleaner and easier to read	Requires indexing (`tuple[i]`)
Less code	More code
Recommended for most cases	Useful for custom index operations

---

When Should You Use `enumerate()`?

Use `enumerate()` when you need to:

- Access both the index and the element.
- Display serial numbers or roll numbers.
- Write cleaner and more readable code.
- Avoid manually managing indexes.

Real-World Example

Suppose you want to display the ranking of players.

```
players = ("Virat", "Rohit", "Hardik", "Jadeja")

for rank, player in enumerate(players, start=1):
    print("Rank", rank, ":", player)
```

Output

```
Rank 1 : Virat
Rank 2 : Rohit
Rank 3 : Hardik
Rank 4 : Jadeja
```

Summary Table

Feature	Description
Function	`enumerate()`
Returns	Index and Element
Default Starting Index	`0`
Custom Starting Index	✅ Yes (`start=`)
Works with Tuples	✅ Yes
Supports Nested Tuples	✅ Yes

Key Points

- `enumerate()` returns both the index and the element during iteration.
- The default starting index is `0`.
- Use the `start` parameter to begin indexing from a different value.
- `enumerate()` makes code cleaner and easier to read than using `range()`.
- It is ideal for displaying serial numbers, roll numbers, rankings, and other indexed data.
- `enumerate()` works with nested tuples and tuple unpacking.

Comparison of Looping Methods

Method	Best Used For
`for` loop	Iterating over elements
`while` loop	Index-based iteration
`range()`	Accessing indexes
`enumerate()`	Getting both index and value

Real-World Example

```
marks = (85, 90, 78, 92, 88)

for mark in marks:
    print("Marks:", mark)
```

Output

```
Marks: 85
Marks: 90
Marks: 78
Marks: 92
Marks: 88
```

Key Points

- Tuples are iterable.
- The `for` loop is the simplest way to iterate through a tuple.
- Use `while` when index-based access is needed.
- Use `range()` to iterate using indexes.
- Use `enumerate()` when both the index and the element are required.
- Nested tuples can be unpacked directly inside a `for` loop.

Tuple Operations

Tuples support several operations that allow you to combine, repeat, search, and compare tuples. The most common tuple operations are:

- `Concatenation` (+)
- `Repetition` (*)
- `Membership Operators` (in, not in)
- `Comparison Operators` (==, !=, <, >, <=, >=)

Concatenation (+)

Concatenation means joining two or more tuples together to create a new tuple.

The `+` **operator** is used for tuple concatenation.

> **Note:** Concatenation does not modify the original tuples. It creates a **new tuple**.

Syntax

```
new_tuple = tuple1 + tuple2
```

Example 1: Join Two Tuples

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)

result = tuple1 + tuple2

print(result)
```

Output

```
(1, 2, 3, 4, 5, 6)
```

Example 2: Join String Tuples

```
fruits = ("Apple", "Banana")
vegetables = ("Potato", "Tomato")

items = fruits + vegetables

print(items)
```

Output

```
('Apple', 'Banana', 'Potato', 'Tomato')
```

Example 3: Multiple Concatenation

```
a = (1, 2)
b = (3, 4)
c = (5, 6)
```

```
result = a + b + c

print(result)
```
Output

```
(1, 2, 3, 4, 5, 6)
```

Example 4: Empty Tuple

```
numbers = (10, 20, 30)

result = numbers + ()

print(result)
```
Output

```
(10, 20, 30)
```

Real-World Example

```
first_sem = ("Math", "Physics")
second_sem = ("Python", "DBMS")

subjects = first_sem + second_sem

print(subjects)
```
Output

```
('Math', 'Physics', 'Python', 'DBMS')
```

Summary Table

Operator	Purpose
`+`	Joins two or more tuples
Result	Creates a new tuple
Original Tuples	Remain unchanged

Key Points

- `+` joins two or more tuples.
- A new tuple is created.
- Original tuples remain unchanged.
- Works with tuples containing any data type.

Repetition (`*`)

The **repetition operator (`*`)** creates a new tuple by repeating the elements of an existing tuple a specified number of times.

> **Note:** The original tuple is not modified.

Syntax

```
new_tuple = tuple_name * number
```

Example 1: Repeat a Tuple

```
numbers = (1, 2, 3)

print(numbers * 3)
```

Output

```
(1, 2, 3, 1, 2, 3, 1, 2, 3)
```

Example 2: Repeat Strings

```
fruits = ("Apple", "Banana")

print(fruits * 2)
```

Output

```
('Apple', 'Banana', 'Apple', 'Banana')
```

Example 3: Repeat Once

```
numbers = (10, 20)

print(numbers * 1)
```

Output

```
(10, 20)
```

Example 4: Repeat Zero Times

```
numbers = (10, 20)

print(numbers * 0)
```

Output

```
()
```

Real-World Example

```
days = ("Mon", "Tue")

schedule = days * 3

print(schedule)
```

Output

```
('Mon', 'Tue', 'Mon', 'Tue', 'Mon', 'Tue')
```

Summary Table

Operator	Purpose
`*`	Repeats a tuple
Result	Creates a new tuple
Original Tuple	Remains unchanged

Key Points

- `*` repeats tuple elements.
- Returns a new tuple.
- The original tuple is unchanged.
- Repeating by `0` returns an empty tuple.#

Membership Operators (`in`, `not in`)

Membership operators are used to check whether an element exists in a tuple.

Python provides two membership operators:

- `in`
- `not in`

`in` Operator

Returns `True` if the element exists in the tuple.

Syntax

```
value in tuple_name
```

Example

```
fruits = ("Apple", "Banana", "Mango")

print("Banana" in fruits)
```

Output

```
True
```

`not in` Operator

Returns `True` if the element does **not** exist in the tuple.

Syntax

```
value not in tuple_name
```

Example

```
fruits = ("Apple", "Banana", "Mango")

print("Orange" not in fruits)
```

Output

```
True
```

More Examples

```
numbers = (10, 20, 30, 40)

print(20 in numbers)
print(100 in numbers)

print(50 not in numbers)
print(30 not in numbers)
```

Output

```
True
False
True
False
```

Real-World Example

```
subjects = ("Python", "DBMS", "OS")

course = "Python"

if course in subjects:
    print("Course Available")
```

Output

```
Course Available
```

Summary Table

Operator	Meaning
`in`	Checks whether an element exists
`not in`	Checks whether an element does not exist

Key Points

- `in` returns `True` if the element exists.
- `not in` returns `True` if the element does not exist.
- Membership operators are commonly used in `if` statements and loops.

Comparison Operators

Comparison operators compare two tuples element by element.

Python starts comparing from the **first element**. If the first elements are equal, it compares the next elements, and so on.

> **Note:** Comparison is performed in **lexicographical order** (similar to dictionary order).

Supported Comparison Operators

Operator	Meaning
`==`	Equal to
`!=`	Not equal to
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to
`>=`	Greater than or equal to

Example 1: Equal To (`==`)

```
a = (10, 20, 30)
b = (10, 20, 30)

print(a == b)
```

Output

```
True
```

Example 2: Not Equal To (`!=`)

```
a = (10, 20, 30)
b = (10, 25, 30)

print(a != b)
```

Output

```
True
```

Example 3: Less Than (`<`)

```
a = (10, 20, 30)
b = (10, 25, 30)

print(a < b)
```

Output

```
True
```

Explanation

- `10 == 10`
- `20 < 25`

Therefore, the result is `True`.

Example 4: Greater Than (`>`)

```
a = (50, 20)
b = (40, 100)

print(a > b)
```

Output

```
True
```

Example 5: Less Than or Equal To (`<=`)

```
a = (10, 20)
b = (10, 20)

print(a <= b)
```

Output

```
True
```

Example 6: Greater Than or Equal To (`>=`)

```
a = (30, 40)
b = (30, 20)

print(a >= b)
```

Output

```
True
```

How Tuple Comparison Works

```
a = (5, 10, 15)
b = (5, 12, 10)
```

Python compares:

Comparison	Result
`5 == 5`	Continue
`10 < 12`	`True`
Remaining elements	Not compared

Real-World Example

```
student1 = (101, "Alice")
student2 = (102, "Bob")

print(student1 < student2)
```

Output

```
True
```

Summary Table

Operator	Example	Result
`==`	`(1,2)==(1,2)`	`True`
`!=`	`(1,2)!=(2,1)`	`True`
`<`	`(1,2)<(1,3)`	`True`
`>`	`(3,2)>(2,5)`	`True`
`<=`	`(1,2)<=(1,2)`	`True`
`>=`	`(3,2)>=(3,1)`	`True`

Key Points

- Python compares tuples element by element.
- Comparison starts from the first element.
- If two elements are equal, Python compares the next elements.
- Tuple comparison follows **lexicographical order**.
- Comparison stops as soon as a difference is found.

Built-in Functions for Tuples

Python provides several **built-in functions** that work with tuples. These functions help you perform common operations such as finding the number of elements, calculating the sum, finding the smallest or largest value, sorting elements, and checking logical conditions.

Built-in functions make your programs **shorter, faster, and easier to read** because you don't need to write the logic manually.

> **Note:** Most built-in functions do **not** modify the original tuple. Instead, they return a new value or a new object.

Common Built-in Functions for Tuples

Function	Purpose
`len()`	Returns the number of elements in a tuple
`min()`	Returns the smallest element
`max()`	Returns the largest element
`sum()`	Returns the sum of all numeric elements
`sorted()`	Returns a new sorted list from the tuple
`any()`	Returns `True` if at least one element is `True`
`all()`	Returns `True` only if all elements are `True`

Why Use Built-in Functions?

Built-in functions help you to:

- Count the number of elements.
- Find the smallest and largest values.
- Calculate the total of numeric elements.
- Sort tuple elements easily.
- Check whether any or all elements satisfy a condition.
- Write cleaner and more efficient Python code.

`len()` Function

The `len()` function is used to find the **number of elements** present in a tuple.

It returns the total count of items stored in the tuple.

> **Note:** `len()` counts every element, including duplicate values.

Syntax

```
len(tuple_name)
```

Example 1: Find the Length of a Tuple

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(len(fruits))
```

Output

```
4
```

Explanation

The tuple contains four elements, so `len()` returns `4`.

Example 2: Tuple of Numbers

```
numbers = (10, 20, 30, 40, 50)

print(len(numbers))
```

Output

```
5
```

Example 3: Empty Tuple

```
empty = ()

print(len(empty))
```

Output

```
0
```

Example 4: Tuple with Duplicate Elements

```
colors = ("Red", "Blue", "Red", "Green")

print(len(colors))
```

Output

```
4
```

Explanation

Duplicate elements are counted individually.

Example 5: Nested Tuple

```
data = (
    (1, 2),
    (3, 4),
    (5, 6)
)

print(len(data))
```

Output

```
3
```

Explanation

The outer tuple contains three elements, each of which is another tuple.

Real-World Example

```
students = (
    "Rahul",
    "Priya",
    "Aman",
    "Neha",
    "Riya"
)

print("Total Students:", len(students))
```

Output

```
Total Students: 5
```

Summary Table

Function	Purpose
`len()`	Returns the number of elements in a tuple

Key Points

- `len()` returns the total number of elements.
- Duplicate elements are counted.
- An empty tuple has a length of `0`.

`min()` Function

The `min()` function returns the **smallest element** in a tuple.

> **Note:** The tuple elements should be comparable. For numeric tuples, it returns the smallest number.

Syntax

```
min(tuple_name)
```

Example 1: Minimum Number

```
numbers = (45, 12, 89, 5, 33)

print(min(numbers))
```

Output

```
5
```

Example 2: String Tuple

```
fruits = ("Mango", "Apple", "Orange")

print(min(fruits))
```

Output

```
Apple
```

Example 3: Negative Numbers

```
numbers = (-5, -20, 15, 8)

print(min(numbers))
```

Output

```
-20
```

Example 4: Single Element Tuple

```
value = (100,)

print(min(value))
```

Output

```
100
```

Real-World Example

```
marks = (85, 72, 91, 66, 78)
```

```
print("Lowest Marks:", min(marks))
```

Output

```
Lowest Marks: 66
```

Summary Table

Function	Purpose
`min()`	Returns the smallest element

Key Points

- Returns the smallest element.
- Works with numbers and strings.
- Elements must be comparable.
- Commonly used to find minimum values.

`max()` Function

The **`max()`** function returns the **largest element** in a tuple.

Syntax

```
max(tuple_name)
```

Example 1: Maximum Number

In [ ]:
```
numbers = (45, 12, 89, 5, 33)

print(max(numbers))
```

Output

```
89
```

Example 2: String Tuple

In [ ]:
```
fruits = ("Mango", "Apple", "Orange")

print(max(fruits))
```

Output

```
Orange
```

Example 3: Negative Numbers

```
numbers = (-5, -20, -2)

print(max(numbers))
```

Output

```
-2
```

Example 4: Single Element Tuple

```
value = (50,)

print(max(value))
```

Output

```
50
```

Real-World Example

```
marks = (85, 72, 91, 66, 78)

print("Highest Marks:", max(marks))
```

Output

```
Highest Marks: 91
```

Summary Table

Function	Purpose
`max()`	Returns the largest element

Key Points

- Returns the largest element.
- Works with numeric and string tuples.
- Useful for finding maximum values.

`sum()` Function

The **`sum()`** function returns the **sum of all numeric elements** in a tuple.

Syntax

```
sum(tuple_name)
```

Example 1: Sum of Numbers

```
numbers = (10, 20, 30, 40)

print(sum(numbers))
```

Output

```
100
```

Example 2: Decimal Numbers

```
values = (2.5, 3.5, 4.0)

print(sum(values))
```

Output

```
10.0
```

Example 3: Negative Numbers

```
numbers = (10, -5, 20)

print(sum(numbers))
```

Output

```
25
```

Example 4: Empty Tuple

```
numbers = ()

print(sum(numbers))
```

Output

```
0
```

Real-World Example

```
expenses = (250, 450, 300, 500)

print("Total Expense:", sum(expenses))
```

Output

```
Total Expense: 1500
```

Summary Table

Function	Purpose
`sum()`	Returns the sum of numeric elements

Key Points

- Works only with numeric tuples.
- Returns `0` for an empty tuple.
- Commonly used in calculations.

`sorted()` Function

The **`sorted()`** function returns a **new sorted list** containing the elements of a tuple.

> **Important:** `sorted()` does **not** return a tuple. It returns a **list**.

Syntax

```
sorted(tuple_name)
```

Example 1: Ascending Order

```
numbers = (40, 10, 30, 20)

print(sorted(numbers))
```

Output

```
[10, 20, 30, 40]
```

Example 2: Descending Order

```
numbers = (40, 10, 30, 20)

print(sorted(numbers, reverse=True))
```

Output

```
[40, 30, 20, 10]
```

Example 3: String Tuple

```
fruits = ("Mango", "Apple", "Orange")

print(sorted(fruits))
```

Output

```
['Apple', 'Mango', 'Orange']
```

Real-World Example

```
marks = (85, 72, 91, 66)

print(sorted(marks))
```

Output

```
[66, 72, 85, 91]
```

Summary Table

Function	Returns
`sorted()`	A new sorted list

Key Points

- Returns a list, not a tuple.
- Original tuple remains unchanged.

- Supports ascending and descending sorting.

`any()` Function

The **`any()`** function returns **`True`** if **at least one element** in the tuple evaluates to **`True`**.

If all elements are **`False`** (or the tuple is empty), it returns **`False`**.

Syntax

```
any(tuple_name)
```

Example 1

```
values = (0, False, 5)

print(any(values))
```

Output

```
True
```

Example 2

```
values = (0, False, "")

print(any(values))
```

Output

```
False
```

Example 3

```
values = ()

print(any(values))
```

Output

```
False
```

Real-World Example

```
attendance = (False, False, True, False)

print(any(attendance))
```

Output

```
True
```

Summary Table

Function	Purpose
`any()`	Returns `True` if at least one element is `True`

Key Points

- Returns `True` if any element is truthy.
- Returns `False` for an empty tuple.
- Commonly used in conditional statements.

`all()` Function

The **`all()`** function returns **`True`** only if **every element** in the tuple evaluates to `True`.

If at least one element is `False`, it returns `False`.

> **Note:** For an empty tuple, `all()` returns `True`.

Syntax

```
all(tuple_name)
```

Example 1

```
values = (1, 2, 3)

print(all(values))
```

Output

```
True
```

Example 2

```
values = (1, 0, 3)

print(all(values))
```

Output

```
False
```

Example 3

```
values = ()

print(all(values))
```

Output

```
True
```

Real-World Example

```
results = (True, True, True)

print(all(results))
```

Output

```
True
```

Summary Table

Function	Purpose
`all()`	Returns `True` only if all elements are `True`

Key Points

- Returns `True` only when every element is truthy.
- Returns `False` if any element is falsy.
- Returns `True` for an empty tuple.
- Commonly used to validate multiple conditions.

Example

In [ ]:
```
numbers = (15, 8, 25, 10)

print("Length :", len(numbers))
print("Minimum:", min(numbers))
print("Maximum:", max(numbers))
print("Sum    :", sum(numbers))
print("Sorted  :", sorted(numbers))
```

Output

```
Length : 4
Minimum: 8
Maximum: 25
Sum    : 58
Sorted  : [8, 10, 15, 25]
```

Explanation

- `len()` counts the elements.
- `min()` returns the smallest value.
- `max()` returns the largest value.
- `sum()` calculates the total.
- `sorted()` returns a new list in ascending order.

Important Notes

- Most built-in functions return a value without changing the original tuple.
- `sorted()` returns a **list**, not a tuple.
- `sum()` works only with numeric values.
- `min()` and `max()` require comparable elements.
- `any()` and `all()` return Boolean values (`True` or `False`).

Summary Table

Function	Return Type
`len()`	Integer
`min()`	Smallest Element
`max()`	Largest Element
`sum()`	Number
`sorted()`	List
`any()`	Boolean
`all()`	Boolean

Key Points

- Built-in functions simplify common operations on tuples.
- They improve code readability and reduce the amount of code you need to write.
- Most functions do not modify the original tuple.
- Different functions return different data types, such as integers, lists, numbers, or Boolean values.
- Learning these functions will help you write more efficient Python programs.

Tuple Methods

Unlike lists, tuples have only two built-in methods because tuples are immutable (their elements cannot be changed after creation).

These methods are used to search and count elements in a tuple.

The two tuple methods are:

- `count()`
- `index()`

`count()` Method

The **`count()`** method is used to count how many times a specified element appears in a tuple.

It returns the **total number of occurrences** of the given value.

> **Note:** If the element is not present in the tuple, `count()` returns `0`.

Syntax

```
tuple_name.count(value)
```

Parameters

Parameter	Description
`value`	The element whose occurrences are to be counted

Return Value

Returns an **integer** representing the number of times the specified element appears in the tuple.

Example 1: Count an Integer

```
numbers = (10, 20, 30, 20, 40, 20)

print(numbers.count(20))
```

Output

```
3
```

Explanation

The value `20` appears **three times** in the tuple.

Example 2: Count a String

```
fruits = ("Apple", "Banana", "Apple", "Orange", "Apple")

print(fruits.count("Apple"))
```

Output

```
3
```

Example 3: Element Not Present

```
numbers = (10, 20, 30)

print(numbers.count(50))
```

Output

```
0
```

Example 4: Boolean Values

```
values = (True, False, True, True)

print(values.count(True))
```

Output

```
3
```

Example 5: Mixed Data Types

```
data = (1, "Python", 1, 3.5, "Python", True)

print(data.count("Python"))
print(data.count(1))
```

Output

2
2

Explanation

- `"Python"` appears twice.
- `1` and `True` are considered equal in Python because `True == 1`.

Real-World Example

```
grades = ("A", "B", "A", "C", "A", "B")

print("Number of A Grades:", grades.count("A"))
```

Output

```
Number of A Grades: 3
```

Summary Table

Method	Purpose	Return Type
`count()`	Counts the occurrences of an element	Integer

Key Points

- `count()` returns the number of occurrences of a value.
- Returns `0` if the element is not found.
- Works with all data types.
- Does not modify the original tuple.

`index()` Method

The **`index()`** method is used to find the **position (index)** of the **first occurrence** of a specified element in a tuple.

> **Note:** If the element appears multiple times, only the **first occurrence** is returned.

If the element is not found, Python raises a **`ValueError`**.

Syntax

```
tuple_name.index(value)
```

Syntax with Start and End Positions

```
tuple_name.index(value, start, end)
```

Parameters

Parameter		Description
`value`		Element to search for
`start`	*(Optional)*	Starting index for the search

Parameter	Description
`end` *(Optional)*	Ending index (exclusive)

Return Value

Returns the **index of the first occurrence** of the specified element.

Example 1: Find the Index

```
fruits = ("Apple", "Banana", "Mango", "Orange")

print(fruits.index("Mango"))
```

Output

```
2
```

Example 2: Duplicate Elements

```
numbers = (10, 20, 30, 20, 40)

print(numbers.index(20))
```

Output

```
1
```

Explanation

Although `20` appears twice, the first occurrence is at index `1`.

Example 3: Using the `start` Parameter

```
numbers = (10, 20, 30, 20, 40)

print(numbers.index(20, 2))
```

Output

```
3
```

Explanation

The search starts from index `2`, so the second occurrence of `20` is found.

Example 4: Using `start` and `end`

```
numbers = (10, 20, 30, 20, 40)

print(numbers.index(20, 1, 4))
```

Output

```
1
```

Example 5: Element Not Found

```
numbers = (10, 20, 30)
```

```
print(numbers.index(50))
```

Output

```
ValueError: tuple.index(x): x not in tuple
```

Explanation

Since `50` is not present, Python raises a `ValueError`.

Example 6: Avoiding the Error

```
numbers = (10, 20, 30)

if 20 in numbers:
    print(numbers.index(20))
```

Output

```
1
```

Explanation

Using the `in` operator before calling `index()` helps prevent a `ValueError`.

Real-World Example

```
subjects = ("Python", "DBMS", "Operating System", "Computer Networks")

print("Python is at position:", subjects.index("Python"))
```

Output

```
Python is at position: 0
```

`count()` vs `index()`

`count()`	`index()`
Counts occurrences of an element	Finds the first occurrence of an element
Returns an integer count	Returns an integer index
Returns `0` if not found	Raises `ValueError` if not found
Searches the entire tuple	Can search within a specified range

---

Summary Table

Method	Purpose	Return Type
`index()`	Returns the index of the first occurrence	Integer

Key Points

- `index()` returns the position of the first occurrence of an element.
- It supports optional `start` and `end` parameters.
- If the element is not found, a `ValueError` is raised.
- Use the `in` operator before `index()` to avoid errors.

- `index()` does not modify the original tuple.

Copying Tuples

Since tuples are immutable, copying them is much simpler than copying lists. You cannot change the elements of a tuple after it is created, so copying mainly creates another reference or another tuple object with the same elements.

There are two common ways to copy a tuple:

- `Assignment (=)`
- `Using the tuple() constructor`

Copying Tuples Using Assignment (`=`)

The simplest way to copy a tuple is by using the **assignment operator (`=`)**.

When you assign one tuple to another variable, both variables refer to the **same tuple object**.

> **Note:** Since tuples are immutable, sharing the same object is completely safe because the tuple cannot be modified.

Syntax

```
new_tuple = original_tuple
```

Example 1: Basic Assignment

```
numbers = (10, 20, 30)

copy_numbers = numbers

print(numbers)
print(copy_numbers)
```

Output

```
(10, 20, 30)
(10, 20, 30)
```

Example 2: Verify Both Variables

```
fruits = ("Apple", "Banana", "Mango")

copy_fruits = fruits

print(fruits == copy_fruits)
```

Output

```
True
```

Explanation

Both variables contain the same elements.

Example 3: Check Object Identity

```
numbers = (1, 2, 3)

copy_numbers = numbers

print(id(numbers))
print(id(copy_numbers))
```

Possible Output

```
140418643223040
140418643223040
```

Explanation

Both variables usually have the **same memory address**, indicating that they reference the same tuple object.

Example 4: Original Tuple Remains Unchanged

```
original = (100, 200, 300)

copied = original

print(original)
print(copied)
```

Output

```
(100, 200, 300)
(100, 200, 300)
```

Real-World Example

```
subjects = ("Python", "DBMS", "OS")

backup = subjects

print("Original:", subjects)
print("Backup:", backup)
```

Output

```
Original: ('Python', 'DBMS', 'OS')
Backup: ('Python', 'DBMS', 'OS')
```

Summary Table

Feature	Description
Copy Method	Assignment (`=`)
Creates New Object	❌ No
Shares Same Object	✅ Yes
Safe for Tuples	✅ Yes

Key Points

- Assignment is the easiest way to copy a tuple.
- Both variables usually refer to the same tuple object.
- No new tuple is created.
- This is safe because tuples are immutable.

Copying Tuples Using `tuple()`

The **`tuple()` constructor** can also be used to create a copy of an existing tuple.

It creates a tuple from another iterable. When a tuple is passed to `tuple()`, the result contains the same elements.

> **Note:** Since tuples are immutable, Python may return the same tuple object as an optimization.

Syntax

```
new_tuple = tuple(original_tuple)
```

Example 1: Copy a Tuple

```
numbers = (10, 20, 30)

copy_numbers = tuple(numbers)

print(copy_numbers)
```

Output

```
(10, 20, 30)
```

Example 2: String Tuple

```
fruits = ("Apple", "Banana", "Mango")

copy_fruits = tuple(fruits)

print(copy_fruits)
```

Output

```
('Apple', 'Banana', 'Mango')
```

Example 3: Check Equality

```
numbers = (5, 10, 15)

copy_numbers = tuple(numbers)

print(numbers == copy_numbers)
```

Output

```
True
```

Example 4: Check Object Identity

```
numbers = (1, 2, 3)

copy_numbers = tuple(numbers)

print(id(numbers))
print(id(copy_numbers))
```

Possible Output

```
140418643223040
140418643223040
```

Explanation

Python may return the same tuple object because tuples are immutable.

Example 5: Create a Tuple from a List

```
numbers = [10, 20, 30]

new_tuple = tuple(numbers)

print(new_tuple)
```

Output

```
(10, 20, 30)
```

Real-World Example

```
months = ("January", "February", "March")

backup = tuple(months)

print("Original:", months)
print("Copy:", backup)
```

Output

```
Original: ('January', 'February', 'March')
Copy: ('January', 'February', 'March')
```

Assignment (`=`) vs `tuple()`

Assignment (`=`)	`tuple()`
Copies the reference	Creates a tuple from an iterable
Usually refers to the same tuple object	May return the same tuple object for an existing tuple
Faster	Slightly more explicit
Best for copying tuples	Useful for converting other iterables into tuples

---

Summary Table

Feature	Description
Function	`tuple()`
Purpose	Creates a tuple from an iterable
Can Copy a Tuple	✅ Yes
Can Convert Lists, Sets, Strings	✅ Yes

Key Points

- `tuple()` creates a tuple from any iterable.
- It can be used to copy an existing tuple.
- For immutable tuples, Python may return the same object internally.
- `tuple()` is especially useful for converting lists, sets, and strings into tuples.
- Both assignment (`=`) and `tuple()` are safe ways to work with tuple copies.

Joining Tuples

Joining tuples means combining two or more tuples to create a new tuple.

Python provides two ways to join tuples:

- Using the `+` operator (Concatenation)
- Using the `*` operator (Repetition)

> **Note:** Since tuples are immutable, joining tuples always creates a new tuple. The original tuples remain unchanged.

Joining Tuples Using `+`

The `+` **operator** is used to join (concatenate) two or more tuples into a single tuple.

It combines the elements of each tuple in the order they appear.

> **Note:** The original tuples are not modified. A new tuple is created.

Syntax

```
new_tuple = tuple1 + tuple2
```

Example 1: Join Two Tuples

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)

result = tuple1 + tuple2

print(result)
```

Output

```
(1, 2, 3, 4, 5, 6)
```

Example 2: Join String Tuples

```
fruits = ("Apple", "Banana")
vegetables = ("Potato", "Tomato")

items = fruits + vegetables

print(items)
```

Output

```
('Apple', 'Banana', 'Potato', 'Tomato')
```

Example 3: Join Multiple Tuples

```
a = (10,)
b = (20, 30)
c = (40, 50)

result = a + b + c

print(result)
```

Output

```
(10, 20, 30, 40, 50)
```

Example 4: Join Empty Tuple

```
numbers = (1, 2, 3)

result = numbers + ()

print(result)
```

Output

```
(1, 2, 3)
```

Example 5: Mixed Data Types

```
tuple1 = (1, "Python")
tuple2 = (3.14, True)

result = tuple1 + tuple2

print(result)
```

Output

```
(1, 'Python', 3.14, True)
```

Real-World Example

```
semester1 = ("Math", "Physics")
semester2 = ("Python", "DBMS")

subjects = semester1 + semester2

print(subjects)
```

Output

```
('Math', 'Physics', 'Python', 'DBMS')
```

Summary Table

Feature	Description
Operator	`+`
Purpose	Joins two or more tuples
Creates New Tuple	✅ Yes
Changes Original Tuple	❌ No

Key Points

- `+` joins two or more tuples.
- A new tuple is created.
- Original tuples remain unchanged.
- Tuples containing different data types can also be joined.

Joining Tuples Using `*`

The `*` **operator** repeats the elements of a tuple a specified number of times.

Although it is primarily known as the **repetition operator**, it can also be used to create a larger tuple by repeating the same tuple multiple times.

> **Note:** The original tuple is not modified. A new tuple containing repeated elements is created.

Syntax

```
new_tuple = tuple_name * number
```

Example 1: Repeat a Tuple Twice

```
numbers = (1, 2, 3)

result = numbers * 2
```

```
print(result)
```
Output

```
(1, 2, 3, 1, 2, 3)
```

Example 2: Repeat Three Times

```
fruits = ("Apple", "Banana")

print(fruits * 3)
```
Output

```
('Apple', 'Banana', 'Apple', 'Banana', 'Apple', 'Banana')
```

Example 3: Repeat Once

```
numbers = (10, 20)

print(numbers * 1)
```
Output

```
(10, 20)
```

Example 4: Repeat Zero Times

```
numbers = (10, 20)

print(numbers * 0)
```
Output

```
()
```

Example 5: Repeat a Single-Element Tuple

```
colors = ("Red",)

print(colors * 4)
```
Output

```
('Red', 'Red', 'Red', 'Red')
```

Real-World Example

```
weekdays = ("Mon", "Tue", "Wed", "Thu", "Fri")

two_weeks = weekdays * 2

print(two_weeks)
```
Output

```
('Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri')
```

`+` vs `*`

`+` Operator	`*` Operator
Joins different tuples	Repeats the same tuple
Combines elements once	Repeats elements multiple times
Requires two or more tuples	Requires one tuple and an integer
Creates a new tuple	Creates a new tuple

Summary Table

Feature	Description
Operator	`*`
Purpose	Repeats a tuple multiple times
Creates New Tuple	✅ Yes
Changes Original Tuple	❌ No

Key Points

- `*` repeats the elements of a tuple.
- The repetition count must be an integer.
- Multiplying by `1` returns the same tuple.
- Multiplying by `0` returns an empty tuple.
- A new tuple is created, while the original tuple remains unchanged.

Nested Tuples

A nested tuple is a tuple that contains one or more tuples as its elements.

Nested tuples are useful for storing grouped or structured data, such as student records, employee information, coordinates, or product details.

> **Note**: Since tuples are immutable, the nested tuple structure cannot be modified after creation.

Creating Nested Tuples

A **nested tuple** is created by placing one or more tuples inside another tuple.

Each inner tuple acts as a single element of the outer tuple.

Syntax

```
nested_tuple = (
    (element1, element2),
    (element3, element4),
    ...
)
```

Example 1: Basic Nested Tuple

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

print(students)
```

Output

```
(('Alice', 20), ('Bob', 21), ('Charlie', 22))
```

Explanation

The outer tuple contains three inner tuples.

Example 2: Nested Tuple of Numbers

```
matrix = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 9)
)

print(matrix)
```

Output

```
((1, 2, 3), (4, 5, 6), (7, 8, 9))
```

Example 3: Mixed Data Types

```
employee = (
    (101, "Rahul"),
    ("Developer", 65000),
    (True, "Full-Time")
)

print(employee)
```

Output

```
((101, 'Rahul'), ('Developer', 65000), (True, 'Full-Time'))
```

Example 4: Multiple Levels of Nesting

```
data = (
    (1, 2),
    (
        (3, 4),
        (5, 6)
    )
)

print(data)
```

Output

```
((1, 2), ((3, 4), (5, 6)))
```

Example 5: Store Student Records

```
students = (
    (101, "Rahul", "A"),
    (102, "Priya", "B"),
    (103, "Aman", "A+")
)

print(students)
```

Output

```
((101, 'Rahul', 'A'), (102, 'Priya', 'B'), (103, 'Aman', 'A+'))
```

Real-World Example

```
products = (
    ("Laptop", 55000),
    ("Mouse", 800),
    ("Keyboard", 1500)
)

print(products)
```

Output

```
(('Laptop', 55000), ('Mouse', 800), ('Keyboard', 1500))
```

Summary Table

Feature	Description
Definition	A tuple containing one or more tuples
Outer Tuple	Stores inner tuples
Inner Tuple	Represents grouped data
Mutable	❌ No

Key Points

- A nested tuple contains tuples inside another tuple.

- Each inner tuple is treated as a single element.
- Nested tuples help organize structured data.
- They are immutable, just like normal tuples.

Accessing Nested Tuples

Elements in a nested tuple are accessed using **multiple indexes**.

The first index selects the **inner tuple**, and the second index selects the **element inside that inner tuple**.

Syntax

```
nested_tuple[outer_index][inner_index]
```

Example 1: Access an Inner Tuple

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

print(students[1])
```

Output

```
('Bob', 21)
```

Explanation

`students[1]` returns the second inner tuple.

Example 2: Access a Specific Element

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

print(students[1][0])
```

Output

```
Bob
```

Explanation

- `students[1]` selects the second tuple.
- `[0]` selects its first element.

Example 3: Access Age

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

print(students[2][1])
```

Output

```
22
```

Example 4: Negative Indexing

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

print(students[-1][-1])
```

Output

```
22
```

Explanation

- `-1` selects the last inner tuple.
- The second `-1` selects its last element.

Example 5: Nested Tuple with Multiple Levels

```
data = (
    (1, 2),
    (
        (3, 4),
        (5, 6)
    )
)

print(data[1][0][1])
```

Output

```
4
```

Explanation

- `data[1]` → `((3, 4), (5, 6))`
- `data[1][0]` → `(3, 4)`
- `data[1][0][1]` → `4`

Example 6: Access Student Records

```
students = (
    (101, "Rahul", "A"),
    (102, "Priya", "B"),
    (103, "Aman", "A+")
```

```
)

print(students[2][1])
print(students[2][2])
```

Output

```
Aman
A+
```

Loop Through a Nested Tuple

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

for student in students:
    print(student)
```

Output

```
('Alice', 20)
('Bob', 21)
('Charlie', 22)
```

Loop with Tuple Unpacking

```
students = (
    ("Alice", 20),
    ("Bob", 21),
    ("Charlie", 22)
)

for name, age in students:
    print(name, age)
```

Output

```
Alice 20
Bob 21
Charlie 22
```

Real-World Example

```
employees = (
    (101, "Rahul", "Developer"),
    (102, "Priya", "Designer"),
    (103, "Aman", "Tester")
)

print(employees[0][1])
print(employees[2][2])
```

Output

```
Rahul
Tester
```

Summary Table

Expression	Result
`data[0]`	First inner tuple
`data[0][1]`	Second element of first inner tuple
`data[-1]`	Last inner tuple
`data[-1][-1]`	Last element of the last inner tuple

Key Points

- Use multiple indexes to access nested tuple elements.
- The first index selects the inner tuple.
- The second index selects an element inside the inner tuple.
- Negative indexing works with nested tuples.
- Nested tuples can be traversed using loops and tuple unpacking.

Tuple Memory Efficiency

One of the major advantages of tuples is that they are **more memory efficient** than lists.

Since tuples are **immutable** (their elements cannot be changed after creation), Python stores them in a more compact way. This allows tuples to consume **less memory** and often makes them **slightly faster** than lists.

> **Note:** If your data does not need to change, using a tuple is usually a better choice than using a list.

Why Do Tuples Use Less Memory?

A tuple uses less memory because:

- It is **immutable**, so Python does not need extra memory for adding, removing, or modifying elements.
- It has a **fixed size**, allowing Python to optimize its storage.
- It does not require additional space for methods related to modification, such as `append()`, `insert()`, or `remove()`.
- Python stores tuple data more compactly than list data.

List vs Tuple Memory

List	Tuple
Mutable	Immutable
Uses more memory	Uses less memory
Can grow or shrink	Fixed size

List	Tuple
Slightly slower	Slightly faster
More overhead	Less overhead

Memory Comparison Example

Python provides the `sys.getsizeof()` function to measure the memory used by an object.

Example 1: Compare Memory Usage

In [ ]:
```
import sys

my_list = [10, 20, 30, 40, 50]
my_tuple = (10, 20, 30, 40, 50)

print("List Memory :", sys.getsizeof(my_list), "bytes")
print("Tuple Memory:", sys.getsizeof(my_tuple), "bytes")
```

Possible Output

```
List Memory : 104 bytes
Tuple Memory: 80 bytes
```

> **Note:** The exact memory values may vary depending on your Python version, operating system, and computer architecture.

Example 2: Large Collection

In [ ]:
```
import sys

numbers_list = list(range(1000))
numbers_tuple = tuple(range(1000))

print(sys.getsizeof(numbers_list))
print(sys.getsizeof(numbers_tuple))
```

Possible Output

```
8056
8040
```

Explanation

Even for larger collections, the tuple generally requires **less memory** than the list.

Example 3: Verify Data Remains the Same

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)

print(my_list)
print(my_tuple)
```

Output

```
[1, 2, 3]
(1, 2, 3)
```

Explanation

Both store the same data, but the tuple uses less memory because it is immutable.

Real-World Example

Suppose a program stores the names of all months in a year.

Since the months never change, using a tuple is more memory-efficient than using a list.

```
months = (
    "January",
    "February",
    "March",
    "April",
    "May",
    "June",
    "July",
    "August",
    "September",
    "October",
    "November",
    "December"
)

print(months)
```

Output

```
('January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
'September', 'October', 'November', 'December')
```

When Should You Use Tuples?

Use tuples when:

- The data should not change.
- Memory efficiency is important.
- You want faster access to fixed data.
- The collection represents constant values (months, days, coordinates, RGB colors, etc.).

Summary Table

Feature	List	Tuple
Mutable	✅ Yes	❌ No
Memory Usage	Higher	Lower
Speed	Slightly Slower	Slightly Faster
Can Add/Remove Elements	✅ Yes	❌ No
Best For	Dynamic Data	Fixed Data

Key Points

- Tuples use less memory than lists because they are immutable.
- Python stores tuples more compactly than lists.
- `sys.getsizeof()` can be used to compare memory usage.
- Memory values may vary across different systems and Python versions.
- Use tuples for fixed data to improve memory efficiency and performance.

Tuple Performance

Tuples are generally **slightly faster** than lists in Python.

Since tuples are **immutable**, Python can optimize their storage and access more efficiently. As a result, operations such as creating, accessing, and iterating through tuples are often faster than performing the same operations on lists.

> **Note:** The performance difference is usually small, but it becomes more noticeable when working with **large amounts of data** or in **performance-critical applications**.

Why Are Tuples Slightly Faster?

Tuples are faster because:

- They are **immutable**, so Python does not need to support operations like adding, removing, or modifying elements.
- Their **fixed size** allows Python to optimize memory allocation.
- They have **less internal overhead** than lists.
- Python can access tuple elements more efficiently.

List vs Tuple Performance

List	Tuple
Mutable	Immutable
Slightly slower	Slightly faster
More memory overhead	Less memory overhead
Best for changing data	Best for fixed data

Performance Comparison Example

The `timeit` module is commonly used to compare the execution time of Python code.

Example 1: Access Speed

In [ ]:
```
import timeit
```

```
list_time = timeit.timeit("[1, 2, 3, 4, 5][2]", number=1000000)
tuple_time = timeit.timeit("(1, 2, 3, 4, 5)[2]", number=1000000)

print("List Time :", list_time)
print("Tuple Time:", tuple_time)
```

Possible Output

```
List Time : 0.0845
Tuple Time: 0.0738
```

> **Note:** The execution time will vary depending on your computer and Python version.

Example 2: Iteration Speed

In [ ]:
```
import timeit

numbers_list = list(range(1000))
numbers_tuple = tuple(range(1000))

list_time = timeit.timeit(
    "for x in numbers_list: pass",
    globals=globals(),
    number=10000
)

tuple_time = timeit.timeit(
    "for x in numbers_tuple: pass",
    globals=globals(),
    number=10000
)

print("List Iteration :", list_time)
print("Tuple Iteration:", tuple_time)
```

Possible Output

```
List Iteration : 0.245
Tuple Iteration: 0.228
```

Example 3: Creating a List vs Tuple

In [ ]:
```
import timeit

list_time = timeit.timeit("[1, 2, 3, 4, 5]", number=1000000)
tuple_time = timeit.timeit("(1, 2, 3, 4, 5)", number=1000000)

print("List Creation :", list_time)
print("Tuple Creation:", tuple_time)
```

Possible Output

```
List Creation : 0.056
Tuple Creation: 0.042
```

When Does Performance Matter?

The performance difference between tuples and lists is usually very small for everyday programs. However, tuples are a better choice when:

- Working with very large datasets.
- Performing millions of iterations.
- Storing constant values that never change.
- Writing performance-critical applications.
- Optimizing memory and execution speed.

Real-World Example

Suppose you want to store the names of all months in a calendar.

Since the months never change, using a tuple is more efficient than using a list.

```
months = (
    "January",
    "February",
    "March",
    "April",
    "May",
    "June",
    "July",
    "August",
    "September",
    "October",
    "November",
    "December"
)

print(months)
```

Output

```
('January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
'September', 'October', 'November', 'December')
```

Memory Efficiency vs Performance

Feature	List	Tuple
Mutable	✅ Yes	❌ No
Memory Usage	Higher	Lower
Access Speed	Fast	Slightly Faster
Iteration Speed	Fast	Slightly Faster
Best Use Case	Frequently changing data	Fixed, read-only data

Summary Table

Feature	Tuple
Performance	Slightly faster than lists

Feature	Tuple
Memory Usage	Lower
Mutability	Immutable
Best For	Read-only or constant data

Key Points

- Tuples are slightly faster than lists because they are immutable.
- Python can optimize tuple storage and access.
- The performance difference is usually small but can matter for large datasets or performance-critical applications.
- Use the `timeit` module to compare execution times.
- Choose tuples when your data does not need to change.

Tuple in Function Returns

Python functions can return **multiple values** at the same time.

Although it appears that multiple values are being returned, Python actually **packs all the returned values into a tuple**.

This feature makes functions simpler, cleaner, and easier to use.

> **Note:** Returning multiple values is one of the most common uses of tuples in Python.

Returning Multiple Values

A function can return more than one value by separating the values with commas.

Python automatically packs these values into a tuple.

Syntax

```
def function_name():
    return value1, value2, value3
```

Example 1: Return Two Values

```
def calculate():
    return 10, 20

result = calculate()

print(result)
print(type(result))
```

Output

```
(10, 20)
<class 'tuple'>
```

Explanation

Python packs `10` and `20` into a tuple before returning them.

Example 2: Return Three Values

```python
def student():
    return "Rahul", 20, "A"

details = student()

print(details)
```

Output

```
('Rahul', 20, 'A')
```

Example 3: Return Different Data Types

```python
def information():
    return 101, "Python", True

data = information()

print(data)
```

Output

```
(101, 'Python', True)
```

Example 4: Return Sum and Product

```python
def calculate(a, b):
    return a + b, a * b

result = calculate(5, 3)

print(result)
```

Output

```
(8, 15)
```

Example 5: Return Multiple Results

```python
def rectangle(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter

result = rectangle(10, 5)

print(result)
```

Output

```
(50, 30)
```

Multiple Assignment

Instead of storing the returned tuple in a single variable, you can directly assign each returned value to a separate variable.

This process is called **multiple assignment** or **tuple unpacking**.

Syntax

```
value1, value2 = function_name()
```

Example 1: Unpack Two Values

```
def calculate():
    return 100, 200

x, y = calculate()

print(x)
print(y)
```

Output

```
100
200
```

Example 2: Unpack Three Values

```
def student():
    return "Rahul", 20, "A"

name, age, grade = student()

print(name)
print(age)
print(grade)
```

Output

```
Rahul
20
A
```

Example 3: Sum and Product

```
def calculate(a, b):
    return a + b, a * b

sum_result, product = calculate(6, 4)
```

```
print(sum_result)
print(product)
```

Output

```
10
24
```

Example 4: Area and Perimeter

```
def rectangle(length, width):
    return length * width, 2 * (length + width)

area, perimeter = rectangle(8, 6)

print(area)
print(perimeter)
```

Output

```
48
28
```

Example 5: Ignore Unwanted Values

```
def student():
    return "Rahul", 20, "A"

name, _, grade = student()

print(name)
print(grade)
```

Output

```
Rahul
A
```

Explanation

The underscore (`_`) is commonly used to ignore values that are not needed.

Real-World Example

Suppose you want a function that returns a student's total marks and percentage.

```
def result():
    total = 450
    percentage = 90.0
    return total, percentage

total_marks, percent = result()

print("Total Marks:", total_marks)
print("Percentage:", percent)
```

Output

```
Total Marks: 450
Percentage: 90.0
```

Returning Multiple Values vs Returning a List

Returning Tuple	Returning List
Immutable	Mutable
Slightly faster	Slightly slower
Uses less memory	Uses more memory
Best for fixed return values	Best when returned data needs modification

---

Summary Table

Feature	Description
Multiple Return Values	Packed into a tuple automatically
Return Type	Tuple
Multiple Assignment	Unpacks returned tuple into variables
Ignore Values	Use `_`

Key Points

- Python functions can return multiple values.
- Multiple returned values are automatically packed into a tuple.
- Multiple assignment (tuple unpacking) assigns each returned value to a separate variable.
- The number of variables should match the number of returned values.
- Use `_` to ignore values that you do not need.
- Returning tuples makes code cleaner, faster, and more readable.

Tuple vs List (Complete Comparison)

Both **tuples** and **lists** are used to store collections of data in Python.

They share many similarities, such as maintaining the order of elements and allowing duplicate values. However, they differ in mutability, performance, memory usage, and use cases.

Choosing the right data structure depends on whether your data needs to change after creation.

Feature Comparison

Feature	Tuple	List
Syntax	`( )`	`[ ]`
Mutable	❌ No	✅ Yes
Ordered	✅ Yes	✅ Yes

Feature	Tuple	List
Allows Duplicates	✅ Yes	✅ Yes
Indexed	✅ Yes	✅ Yes
Supports Slicing	✅ Yes	✅ Yes
Nested Collections	✅ Yes	✅ Yes
Memory Efficient	✅ Yes	❌ No
Faster	✅ Slightly Faster	❌ Slightly Slower
Can Grow/Shrink	❌ No	✅ Yes
Can Modify Elements	❌ No	✅ Yes
Built-in Methods	2 (`count()` , `index()`)	Many (`append()` , `insert()` , `remove()` , `sort()` , etc.)
Suitable as Dictionary Key	✅ Yes (if all elements are immutable)	❌ No
Best For	Fixed Data	Dynamic Data

Key Points

- Tuples are immutable, while lists are mutable.
- Both maintain the insertion order and allow duplicate values.
- Tuples generally use less memory and are slightly faster than lists.
- Lists provide many methods for modifying data.
- Use tuples for fixed, read-only data and lists for data that changes frequently.

Real-World Applications of Tuples ⭐

Tuples are widely used in Python whenever the data is **fixed** and **should not change**.

Since tuples are **immutable**, **memory-efficient**, and **slightly faster** than lists, they are ideal for storing constant or read-only data.

Below are some of the most common real-world applications of tuples.

---

1. Coordinates (x, y)

Coordinates represent a fixed position on a graph or map.

Since the values usually do not change, tuples are commonly used.

Example

```
point = (10, 25)

print("X Coordinate:", point[0])
print("Y Coordinate:", point[1])
```

Output

```
X Coordinate: 10
Y Coordinate: 25
```

Explanation

The tuple stores the **x-coordinate** and **y-coordinate** of a point.

---

2. RGB Colors (255, 0, 0)

Computers represent colors using **RGB (Red, Green, Blue)** values.

Each color consists of three fixed values ranging from **0 to 255**.

Tuples are perfect because RGB values should not change accidentally.

Example

```
red = (255, 0, 0)
green = (0, 255, 0)
blue = (0, 0, 255)

print(red)
print(green)
print(blue)
```

Output

```
(255, 0, 0)
(0, 255, 0)
(0, 0, 255)
```

---

3. Database Records

Database records often contain related information.

Each record can be represented as a tuple.

Example

```
student = (101, "Rahul", "CSE", 8.9)

print(student)
```

Output

```
(101, 'Rahul', 'CSE', 8.9)
```

Explanation

The tuple stores:

- Student ID
- Name
- Department

- CGPA

Multiple Database Records

```
students = (
    (101, "Rahul", "CSE"),
    (102, "Priya", "ECE"),
    (103, "Aman", "IT")
)

print(students)
```

Output

```
((101, 'Rahul', 'CSE'), (102, 'Priya', 'ECE'), (103, 'Aman', 'IT'))
```

---

4. GPS Location

GPS coordinates consist of **latitude** and **longitude**.

These two values naturally form a tuple.

Example

```
location = (28.6139, 77.2090)

print("Latitude :", location[0])
print("Longitude:", location[1])
```

Output

```
Latitude : 28.6139
Longitude: 77.2090
```

---

5. Returning Multiple Values

Python automatically returns multiple values as a tuple.

This is one of the most common real-world uses of tuples.

Example

```
def calculate(a, b):
    return a + b, a * b

sum_value, product = calculate(5, 3)

print(sum_value)
print(product)
```

Output

```
8
15
```

Explanation

The function returns two values, which are packed into a tuple and then unpacked into two variables.

---

6. Configuration Settings

Applications often have settings that remain constant while the program is running.

These settings can be stored in tuples.

Example

```
database = (
    "localhost",
    3306,
    "admin"
)

print(database)
```

Output

```
('localhost', 3306, 'admin')
```

Another Example

```
window_size = (1920, 1080)

print(window_size)
```

Output

```
(1920, 1080)
```

The screen resolution remains fixed until the user changes it.

More Real-World Uses of Tuples

Application	Example
Mathematical Coordinates	`(10, 20)`
RGB Colors	`(255, 0, 0)`
GPS Location	`(28.6139, 77.2090)`
Student Records	`(101, "Rahul", "CSE")`
Employee Records	`(1001, "Amit", "Manager")`
Function Returns	`(sum, product)`
Screen Resolution	`(1920, 1080)`
Database Configuration	`("localhost", 3306, "admin")`
Date	`(17, 7, 2026)`

Application	Example
Time	`(10, 30, 45)`

Why Use Tuples in These Applications?

Reason	Benefit
Immutable	Prevents accidental modification
Memory Efficient	Uses less memory
Faster	Slightly faster than lists
Ordered	Preserves the order of values
Supports Multiple Data Types	Stores different types together
Ideal for Fixed Data	Best for constant information

Summary Table

Application	Example
Coordinates	`(x, y)`
RGB Colors	`(255, 0, 0)`
Database Records	`(101, "Rahul", "CSE")`
GPS Location	`(latitude, longitude)`
Returning Multiple Values	`return a, b`
Configuration Settings	`("localhost", 3306, "admin")`

Key Points

- Tuples are best for storing **fixed, read-only data**.
- Coordinates, RGB colors, GPS locations, and configuration settings naturally fit tuple structures.
- Database records can be represented as tuples, especially when the data is not modified.
- Python functions automatically return multiple values as tuples.
- Tuples improve memory efficiency and help prevent accidental changes to important data.

Common Errors

- ❌ Forgetting the comma in a single-element tuple: `(5)` instead of `(5,)`
- ❌ Trying to modify tuple elements (`t[0] = 10`)
- ❌ Using list methods like `append()` or `remove()` on tuples
- ❌ Accessing an index that doesn't exist (`IndexError`)
- ❌ Unpacking into the wrong number of variables
- ❌ Confusing lists (`[]`) with tuples (`()`)

Best Practices

- ✅ Use tuples for **fixed or read-only data**.
- ✅ Use lists when the data needs to change.
- ✅ Use meaningful variable names (e.g., `student`, `location`, `rgb_color`).
- ✅ Use tuple unpacking to make code cleaner.
- ✅ Use tuples for function return values.
- ✅ Use tuples as dictionary keys only if all elements are immutable.
- ✅ Add a comma for single-element tuples: `(value,)`.
- ✅ Use parentheses for better readability, even though they are sometimes optional.

💻 Coding Questions (10)

1. Create a tuple containing five integers and print it.
2. Create a single-element tuple and print its type.
3. Write a program to access and print the third element of a tuple.
4. Write a program to find the length of a tuple using `len()`.
5. Write a program to count how many times an element appears in a tuple.
6. Write a program to find the index of a given element in a tuple.
7. Write a program to join two tuples using the `+` operator.
8. Write a program to unpack a tuple into separate variables and print each variable.
9. Write a function that returns the sum and product of two numbers using a tuple.
10. Write a program to create a nested tuple of student records and display each student's name and marks.

💻 Practice Programs (10)

1. Create a tuple of five fruits and print all elements.
2. Create a tuple of numbers and print the first and last elements.
3. Write a program to check whether an element exists in a tuple.
4. Write a program to find the maximum and minimum values in a tuple.
5. Write a program to concatenate two tuples.
6. Write a program to repeat a tuple three times using the `*` operator.
7. Write a program to iterate through a nested tuple and print all elements.
8. Write a program to compare the memory usage of a list and a tuple using `sys.getsizeof()`.
9. Write a program to compare the execution time of a list and a tuple using the `timeit` module.

10. Write a program that returns multiple values from a function and unpacks them into separate variables.

📌 Tuple Cheat Sheet

```
# Create a tuple
t = (1, 2, 3)

# Single-element tuple
t = (5,)

# Empty tuple
t = ()

# Access elements
t[0]
t[-1]

# Slicing
t[1:3]

# Length
len(t)

# Count occurrences
t.count(2)

# Find index
t.index(3)

# Concatenate tuples
t1 + t2

# Repeat tuple
t * 3

# Membership
2 in t
5 not in t

# Built-in functions
min(t)
max(t)
sum(t)
sorted(t)
any(t)
all(t)

# Packing
student = ("Rahul", 20, "A")

# Unpacking
name, age, grade = student

# Nested tuple
students = (
    (101, "Rahul"),
    (102, "Priya")
```

```
)

# Return multiple values
def calculate(a, b):
    return a + b, a * b

# Copy tuple
copy = tuple(t)
copy = t

# Memory size
import sys
sys.getsizeof(t)

# Performance
import timeit
```

📝 Summary

- A **tuple** is an ordered, immutable collection in Python.
- Tuples are created using **parentheses** `()`.
- They allow **duplicate values** and support **multiple data types**.
- Elements are accessed using **indexing** and **slicing**.
- Tuples cannot be modified after creation.
- Tuples support only **two methods**: `count()` and `index()`.
- They can be joined using `+` and repeated using `*`.
- Nested tuples allow storing structured data.
- Tuples use **less memory** and are **slightly faster** than lists.
- Functions return multiple values as tuples.
- Use tuples for **fixed, read-only data** such as coordinates, RGB colors, GPS locations, and configuration settings.